



הפקולטה להנדסה ומדעי המחשב
החוג להנדסת חשמל ואלקטרוניקה

פרויקט גמר באלקטרוניקה

מימוש בחומרה של אלגוריתם ההצפנה Salsa

Hardware Implementation of The Encryption Algorithm Salsa

אייר תשפ"ו
ירושלים

מגיש : ניסן-משה שלמוב
מנחה : מר אורי שטרו

תודות

ראשית, ברצוני להודות לבורא עולם, החונן לאדם דעת ומלמד לאנוש בינה, אשר בחסדיו הרבים הביאני עד הלום ואפשר לי להגיע להישג זה.

ברצוני להודות מקרב לב למשפחתי האהובה: לשירה, אשתי היקרה, על הסבלנות, התמיכה, וההשראה לאורך כל הדרך. להורי שיחיו, על העידוד הבלתי נגמר, הערכים, הדאגה, העזרה והחינוך הראוי להערכה. לחמי וחמותי היקרים על הדאגה והעזרה לאורך הדרך. ולילדי המתוקים - עדי, נטע ואיתן, מקור האור והשמחה שלי שליוו אותי לאורך כל התואר. תודה גם לכל שאר בני המשפחה, שתמיכתם, אמונם ואהבתם נתנו לי את הכוח להתמיד גם ברגעים מאתגרים וקשים.

בתחום האקדמי, אני רוצה להביע תודה מיוחדת לכל צוות המרכז האקדמי לב, שליוו אותי לאורך השנים ותרמו להתפתחותי הן מבחינה מקצועית והן מבחינה אישית. בראש ובראשונה מודה למנחה הפרויקט שלי, מר אורי שטרו, על ההדרכה, ההקשבה, וההכוונה המקצועית והאישית. תודה לד"ר דוד מרסלו, על התמיכה, ההקשבה, העזרה והדוגמה האישית בשילוב חיים רוחניים ומדעיים. לפרופסור בני מיליגרום, ראש החוג, על זמינותו והקשבתו לכל עניין, גם הקטן ביותר, ועל היחס המסור למחלקה ולסטודנטים. ולגברת ענת, מזכירת החוג, שתמיד עמדה לרשותנו וסייעה בכל צורך, כמו אמא חמה ומטפחת.

תודתי נתונה גם לצוות בית המדרש, על העמקה בלימוד, העצות החכמות והתמיכה ברגעים מאתגרים - לרב יוסף צבי רימון, הרב שי, והרב גלינסקי.

לצד ההכוונה האקדמית והרוחנית, אני רוצה להודות לחבריי לכיתה על הלימוד הפורה, ההקשבה והרצון ללמוד ולהתפתח יחד. תודה מיוחדת לדביר בכר, השותף שלי לאורך כל הדרך, על החברות האמיתית, ועל השעות הארוכות של למידה משותפת ומהנה. תודה לשמואל אפלבוים ולנועם כלפון על חברות, הקשבה, ותמיכה לאורך הדרך.

ולבסוף, תודה לכל מי שתמך בדרך זו, בין באופן ישיר ובין באופן עקיף - כל אחד מכס תרם בדרך ייחודית משלו להצלחתי, ולי היה העונג לדעת שאני מוקף בתמיכה, אהבה והשראה לאורך כל הדרך.

תוכן עניינים

6	תקציר
8	א. מבוא
8	א.1. מטרת העבודה
8	א.2. חשיבות העבודה
8	א.3. תרומה למדע ולציבור
9	ב. רקע תאורטי וסקירת ספרות
9	ב.1. קריפטוגרפיה והצפנה
9	ב.2. אלגוריתם Salsa20
10	ב.2.א. פונקציית Quarter Round
11	ב.2.ב. פונקציית Row Round
11	ב.2.ג. פונקציית Column Round
12	ב.2.ד. פונקציית Double Round
12	ב.2.ה. פונקציית הגיבוב Salsa20
13	ב.3. רקע על ה FPGAs השונים
13	ב.3.א. Basys 3 – Xilinx AMD
13	ב.3.ב. DE10-Lite - Intel FPGA
13	ב.4. פרוטוקול התקשורת UART
15	ג. פיתוח ושיטות
15	ג.1. הקדמה לתכנון ואפיון המערכת
16	ג.1.א. Endianess - עקרון ה
16	ג.1.ב. לוגיקה קומבינטורית וסדרתית
17	ג.1.ג. שיטת Unrolling
17	ג.2. תכנון החומרה
17	ג.2.א. constant_select
18	ג.2.ב. תכנון אלגוריתם ההצפנה
21	ג.2.ג. תכנון יחידת הרגיסטרים
23	ג.2.ד. קובץ ההגדרות Salsa_Defines
23	ג.2.ה. תכנון UART
23	ג.2.ו. תכנון מכונת המצבים Salsa_FSM
28	ג.3. תכנון Salsa_top והגדרות constraints
28	ג.3.א. Basys3 - salsa_top
29	ג.3.ב. DE10-Lite - salsa_top
31	ג.4. Python Scripts
31	ג.4.א. salsa20 script
32	ג.4.ב. text2bytes script
32	ג.4.ג. salsa_pyserial_uart script
32	ג.4.ד. auto_and_menaul_test script
33	ג.5. Perl Script
33	ג.5.א. text_vector_to_json script
34	ג.6. Tools
34	ג.6.א. ModelSim \ QuestaSim - סימולציה ואימות פונקציונלי

34	Xilinx / AMD FPGA Vivado - פיתוח עבור
34	Intel (Altera) FPGA Quartus - פיתוח עבור
35	תוצאות
35	Basys3 – תוצאות
35	First Implementation
38	Iterative Hash VS Unrolling Hash
39	Second Implementation
41	Third Implementation
42	Vivado PLL IP
43	The Last Implementation
45	אינטגרציית המערכת
47	DE10-Lite – תוצאות
47	Implementation
49	אינטגרציה
51	דיונים
51	Endianness - סוגיית ה-
51	תפקידן של המילים הקבועות
52	מסקנות וסיכום
53	ספרות
54	נספחים
54	קישורים רלוונטיים
55	נספח א' – פלט פונקציות salsa20 ו-text2bytes
58	נספח ב' – פלט פונקציית perl
65	נספח ג' - XDC Basys3
68	נספח ד' - SDC DE10-Lite
69	נספח ה' - QSF DE10-Lite

10	1- איור ב.1 - תיאור סכמתי של הפונקציה Quarter Round
11	2- איור ב.2 - דיאגרמת הבלוקים של פונקציית Row Round
12	3- איור ב.3 - דיאגרמת הבלוקים של פונקציית Column Round
12	4- איור ב.4 - דיאגרמת הבלוקים של פונקציית Double Round
14	5- איור ב.5 - תיאור מבנה חבילת נתונים בתקשורת UART
15	6- איור ג.1 - דיאגרמת בלוקים עילית של המערכת
15	7- איור ג.2 - חלוקת העבודה למשימות
16	8- איור ג.3 - שיטות ה- Endianness
16	9- איור ג.4 - לוגיקה סדרתית VS לוגיקה קומבינטורית
17	10- איור ג.5 - דיאגרמת בלוקים constant select
18	11- איור ג.6 - דיאגרמת בלוקים salsa_hashing_wrap
19	12- איור ג.7 - דיאגרמת בלוקים salsa_hash_interative_wrapper
19	13- איור ג.8 - דיאגרמת בלוקים hash_interative_with_d_r_c
20	14- איור ג.9 - דיאגרמת המצבים של d_r_c_fsm
20	15- איור ג.10 - דיאגרמת המצבים של hash_interative_fsm
22	16- איור ג.11 - דוגמת דיאגרמת זמנים של סיגנלי Pulse ו- Sticky במערכת
22	17- איור ג.12 - דיאגרמת בלוקים של salsa_registers
25	18- איור ג.13 - דיאגרמת הבעות של מכונת המצבים salsa_FSM
26	19- איור ג.14 - דוגמת גלים חלקית של מכונת המצבים salsa_fsm במצב RX
27	20- איור ג.15 - דוגמת גלים חלקית של מכונת המצבים salsa_fsm במצב TX
28	21- איור ג.16 - Basys3 component description מתוך דפי הנתונים
29	22- איור ג.17 - DE10-Lite Layout and components מתוך דפי הנתונים
30	23- איור ג.18 - Arduino UNO3 description מתוך דפי הנתונים
30	24- איור ג.19 - חיבוריות בין FTDI לבורד
35	25- איור ד.1 - תקציר first implementation
35	26- איור ד.2 - הודעות מערכת Vivado בניסוי הראשון
36	27- איור ד.3 - Dataflow Hierarchy בניסוי הראשון
36	28- איור ד.4 - RTL Scematic לפני סינטזה בניסוי הראשון
37	29- איור ד.5 - RTL Scematic אחרי סינטזה בניסוי הראשון
37	30- איור ד.6 - Implementation Device בניסוי הראשון
38	31- איור ד.7 - Utilization report Unrolling hash
38	32- איור ד.8 - Utilization report Iterative hash
39	33- איור ד.9 - היררכיית הפרויקט בניסיון השני
39	34- איור ד.10 - Implementation Device בניסוי השני
40	35- איור ד.11 - תקציר ניסיון שני
40	36- איור ד.12 - Timing Report ניסיון שני
41	37- איור ד.13 - היררכיית הפרויקט בניסיון השלישי
41	38- איור ד.14 - תקציר ניסיון שלישי
42	39- איור ד.15 - הגדרות PLL IP של Vivado
43	40- איור ד.16 - היררכיית הפרויקט בניסיון האחרון
43	41- איור ד.17 - תקציר הפרויקט הסופי
44	42- איור ד.18 - תקציר דוח הזמנים הסופי
44	43- איור ד.19 - Implementation device הפרויקט הסופי
45	44- איור ד.20 - קטע טקסט מתוך הארי פוטר ואבן החכמים
45	45- איור ד.21 - קטע הטקסט Harry potter המוצפן
45	46- איור ד.22 - קטע הטקסט Harry potter המפוענח
46	47- איור ד.23 - השוואה בין הטקסט Harry potter המפוענח לטקסט המקורי (html)
47	48- איור ד.24 - תקציר הפרויקט DE10-Lite
47	49- איור ד.25 - RTL schmatic DE10-Lite
47	50- איור ד.26 - דוח זמנים DE10-Lite
48	51- איור ד.27 - d_r_c_interaitve FSM by Quartus
48	52- איור ד.28 - hash_interative FSM by Quartus
48	53- איור ד.29 - chip planner DE10-Lite
49	54- איור ד.30 - קטע טקסט מתוך פניני הלכה
49	55- איור ד.31 - קטע פניני הלכה המוצפן
49	56- איור ד.32 - קטע פניני הלכה המפוענח
50	57- איור ד.33 - השוואה בין הקטע פניני הלכה המפוענח לקטע המקורי (html)

תקציר

בעידן שבו כמויות המידע הרגיש גדלות בקצב מואץ - בתחומים אזרחיים, תעשייתיים וביטחוניים, גוברת הדרישה לפתרונות הצפנה מהירים, חסכוניים ואמינים. אלגוריתם ההצפנה Salsa20, שפותח על ידי דניאל ברנשטיין, מהווה אחד מצפני הזרם המודרניים היעילים ביותר, ומציע שילוב של מהירות גבוהה, פשטות מבנית ועמידות קריפטוגרפית. מטרת פרויקט זה הייתה לממש את האלגוריתם ברמת חומרה על גבי מספר FPGAs, במטרה לאפשר יישום הצפנה בעל ביצועים גבוהים תוך ניצול אופטימלי של משאבי החומרה והגדלת בטיחות המידע.

הגישה שננקטה בפרויקט כללה תכנון ארכיטקטורה מלא בשפת Verilog, תוך פירוק האלגוריתם למרכיביו הלוגיים: פעולות QuarterRound, סבבי עמודות ושורות, יצירת Keystream ושילובו עם נתוני הקלט. המימוש נבנה באופן מודולרי כדי להבטיח גמישות, יכולת הרחבה ונוחות בדיקות. הבדיקה והאימות בוצעו באמצעות סימולציות RTL בכלי ModelSim/Questa, תוך שימוש בוקטורי בדיקה ובהשוואת התוצאות מול יישום התוכנה של Salsa20. בנוסף, פותחה מעטפת בדיקה מבוססת UART ו-Python/Perl לצורך תקשורת עם ה-FPGAs השונים והרצת וקטורי בדיקה בזמן אמת.

תוצאות הסימולציה והבדיקה החומרתית הצביעו על התאמה מלאה בין המימוש לבין הפלטים הצפויים של האלגוריתם לכל שלבי ההצפנה. המימוש השיג זמן חישוב קצר לכל בלוק, ניצול משאבי חומרה נמוך יחסית, ושמירה על קצב העברה גבוה. הצלבת התוצאות בין סימולציות תוכנה, סימולציות RTL ובדיקות FPGA בפועל הוכיחה את תקפות המימוש ואת יציבותו התפעולית.

למימוש זה תרומה יישומית ותיאורטית כאחד: מצד אחד, הוא מאפשר שילוב של מנגנון הצפנה מהיר במערכות משובצות, מכשירים רפואיים, רכיבי תקשורת ומערכות ביטחון שבהן נדרשת הצפנה חסכונית ומהירה. מצד שני, הוא מספק דוגמה מלאה לתהליך תכנון, מימוש ואימות של פרוטוקול קריפטוגרפי מתקדם ברמת RTL החל מהארכיטקטורה ועד לבדיקות חומרה. הפרויקט מהווה תשתית לפיתוחים עתידיים כגון הרחבת המימוש ל-Salsa20/8, Salsa20/12, או מעבר למשפחת האלגוריתמים המשופרת ChaCha ו-XChaCha.

Abstract

In an era where the volume of sensitive information is rapidly increasing - across civilian, industrial, and defense domains - the demand for fast, efficient, and reliable encryption solutions continues to grow. The Salsa20 encryption algorithm, developed by Daniel Bernstein, is considered one of the most efficient modern stream ciphers, offering a combination of high speed, structural simplicity, and strong cryptographic robustness. The goal of this project was to implement the algorithm in hardware across multiple FPGA devices, enabling high-performance encryption while optimizing hardware resource usage and enhancing the overall security of the system.

The approach taken in this project included the full architectural design of the algorithm in Verilog, decomposing Salsa20 into its logical components: QuarterRound operations, column and row rounds, keystream generation, and integration with input data. The implementation was built in a modular manner to ensure flexibility, scalability, and ease of testing. Verification was performed using RTL simulations in ModelSim/Questa, utilizing known test vectors and comparing the results against the reference software implementation of Salsa20. In addition, a dedicated testing framework was developed using UART communication and Python/Perl scripts to interface with the various FPGAs and execute test vectors in real time.

Simulation and hardware testing results demonstrated full consistency between the hardware implementation and the expected outputs of the algorithm across all encryption stages. The design achieved short computation time per block, relatively low hardware resource utilization, and maintained a high throughput rate. Cross-validation between software simulation, RTL simulation, and on-FPGA testing confirmed the correctness, stability, and operational reliability of the implementation.

This implementation offers both practical and theoretical contributions: on one hand, it enables the integration of a high-speed encryption mechanism in embedded systems, medical devices, communication modules, and defense systems where efficient and secure encryption is essential. On the other hand, it provides a complete demonstration of designing, implementing, and verifying an advanced cryptographic protocol at the RTL level - from the architectural phase through hardware validation. The project also forms a foundation for future developments, such as extending the design to Salsa20/12, Salsa20/8, or transitioning to the improved ChaCha and XChaCha algorithm family.

א. מבוא

א.1. מטרת העבודה

מטרת הפרויקט היא תכנון, מימוש ואימות של אלגוריתם ההצפנה Salsa20 על גבי פלטפורמות FPGA מגוונות, תוך התמקדות באופטימיזציות ביצועים, ניצול מיטבי של משאבי חומרה והשגת איזון בין מהירות עיבוד, צריכת הספק ושטח לוגי. במסגרת העבודה ייבחנו ארכיטקטורות שונות למימוש האלגוריתם.

אלגוריתם ההצפנה Salsa20 הוא צופן זרם שפותח בשנת 2005 על ידי פרופ' דניאל ברנשטיין באוניברסיטת אילינוי בשיקגו. אלגוריתם Salsa הוצע עבור פרויקט eSTREAM של הארגון האירופאי ECRYPT שרצה למצוא צופני זרם יעילים לשימושים עתידיים. הפרויקט פורסם ב-2004 והתמקד בפיתוח אלגוריתמים בעלי ביצועים גבוהים ועמידות בפני התקפות קריפטוגרפיות. הפרויקט חילק את האלגוריתמים לשתי קטגוריות: צופני זרם בעלי ביצועים גבוהים למעבדים רגילים וצופני זרם המיועדים למערכות משובצות (Embedded Systems) עם משאבים מוגבלים.

Salsa20 הוא אחד האלגוריתמים הידועים ביותר שעלו בפרויקט eSTREAM שזכה לפופולריות רבה בזכות הביצועים הגבוהים והבטיחות שלו ואף נבחר כאחד מארבעת האלגוריתמים המובילים. הגרסה הנפוצה ביותר של Salsa היא Salsa20 המשתמשת ב 10 סיבובים (10 כפול 2 – כל סיבוב מתבצע משני כיוונים שונים). קיימות גרסאות נוספות של 4 ו 6 סיבובים כלומר Salsa8 ו Salsa12 בהתאמה – המיועדות למערכות בעלות משאבי חישוב מוגבלים.

בפרויקט זה האלגוריתם מומש על רכיבי FPGA שונים בצורה אופטימלית ככל שניתן כדי להשיג מהירות תגובה גבוהה, עמידות גבוהה בפני התקפות צד (Side-Channel Attacks), וניצול יעיל של משאבי החומרה. הפרויקט כולל תכנון ארכיטקטורת חומרה, כתיבת קוד בשפת Verilog, ביצוע סימולציות לבדיקת נכונות ויעילות, כתיבת סקריפטים לתקשורת ולמערכת הכללית של האלגוריתם, ופריסת האלגוריתם על גבי FPGA של חברות הטכנולוגיה – Xilinx, AMD, Intel.

א.2. חשיבות העבודה

הצורך באבטחת מידע במערכות זמן אמת ובמערכות משובצות הולך וגובר, לאור העלייה בכמות המידע הרגיש המועבר ברשתות ובמערכות תקשורת. יישום מהיר ויעיל של Salsa20 ברמת החומרה יאפשר למערכות משובצות להצפין ולפענח מידע בזמן אמת, תוך שמירה על רמת ביצועים גבוהה ועמידות בפני התקפות.

בנוסף, מימוש חומרה של אלגוריתם ההצפנה מאפשר יתרונות רבים בהשוואה למימוש בתוכנה

- מהירות גבוהה יותר.
 - חסכון באנרגיה - צריכת חשמל נמוכה יותר בשל העובדה שחומרה מותאמת מבצעת את החישובים ביעילות רבה יותר.
 - שטח חומרה אופטימלי - התאמת הארכיטקטורה למבנה של ה FPGA מאפשרת חיסכון במשאבים לוגיים וזיכרון.
- יתר על כן, תהליך ההצפנה במימוש חומרתי מתבצע ישירות בתהליכים פיזיקליים ישירים (טרנזיסטורים) ולא בצורה וירטואלית, לכן נוכל לקבל עמידות גבוהה מפני התקפות.

א.3. תרומה למדע ולציבור

פרויקט זה תורם רבות הן למדע והן לציבור.

- תרומה צבאית/ביטחונית – ההצפנה ופענוח של מידע רגיש ביותר הדורש מהירות אופטימלית בזמן אמת.
- תרומה סיעודית – מערכות אבטחה ברפואה הנדרשות לשמירת נתונים אישיים ברמה הגבוהה ביותר.
- תרומה למדע – בעת מימוש המצב האופטימלי – Salsa20, ניתן יהיה לבחון ולנתח בצורה מדויקת ויעילה את מגבלותיו ויתרונותיו של האלגוריתם. כמו כן, השוואה בין צורות פריסה שונות יתרום להבנת ההשפעות של פרמטרים ארכיטקטוניים שונים על ביצועי האלגוריתם.

ככל שהטכנולוגיה מתקדמת, ישנה עלייה ניכרת בכמות המידע האישי המועבר ברשת והרצון של כל אדם זה שמירה על הפרטיות שלו, לכן פרויקט זה יכול לתרום רבות למאמץ ולשמור על פרטיותנו בכך שחוקר את האלגוריתם בצורה הכי אופטימלית שניתן.

ב. רקע תאורטי וסקירת ספרות

ב.1. קריפטוגרפיה והצפנה

קריפטוגרפיה היא תחום במדעי המחשב והמתמטיקה העוסק בהגנה על מידע באמצעות שיטות מתמטיות. המטרה העיקרית של הקריפטוגרפיה היא להבטיח סודיות, שלמות, אימות וזהות בעת תקשורת או אחסון מידע.

ישנם מספר סוגי הצפנה, לכל סוג יש את היתרונות והחסרונות שלו.

- הצפנה סימטרית – (Symmetric Encryption) בשיטה זו משתמשים באותו מפתח להצפנה ולפענוח. הדוגמה הקלאסית היא אלגוריתם AES. [1]
- הצפנה אסימטרית – (Asymmetric Encryption) הצפנה עם שני מפתחות: מפתח ציבורי להצפנה ומפתח פרטי לפענוח. דוגמה ידועה היא אלגוריתם RSA. [1]
- הצפנת זרם – (Stream Cipher) הצפנה שפועלת על סיביות או בתים בודדים של נתונים, בניגוד להצפנה בבלוקים. דוגמה בולטת היא Salsa20. [2]
- הצפנת בלוקים – (Block Cipher) הצפנה המחלקת את הנתונים לבלוקים ומצפינה כל בלוק בנפרד. [2]

ב.2. אלגוריתם Salsa20

Salsa20, הוא צופן זרם שתוכנן על ידי דניאל ברנשטיין (Daniel J. Bernstein) בשנת 2005 במסגרת פרויקט eSTREAM. הוא נחשב לצופן יעיל במיוחד, מהיר מאוד ועמיד בפני התקפות קריפטוגרפיות שונות. מאפיינים עיקריים של האלגוריתם [3]:

- יעילות גבוהה - פועל במהירות גבוהה גם על מעבדים פשוטים.
- עמידות בפני התקפות - לא נמצאו חולשות משמעותיות.
- תמיכה באורכי מפתח שונים - דרך כלל 128 או 256 ביטים.
- מבוסס על טרנספורמציות של XOR, הזזות והחלפות ביטים – ללא שימוש בטבלאות חיפוש (S-boxes), מה שמשפר את הביצועים.

האלגוריתם בעצם מייצר חתימה בגודל 512 ביטים (64 בתים = 16 מילים) ע"י הפעלת פונקציית הגיבוב מספר פעמים על המטריצה התחלתית, מטריצה 4x4 של 16 מילים כמתואר בטבלה (1) המורכבת מארבעת הרכיבים הבאים:

1. מפתח (Key) בגודל 256 ביטים (32 בתים = 8 מילים) – במידה ועובדים עם מפתח בגודל 128 ביטים משרשרים את אותות מפתח פעמיים.
2. מספר אקראי (Nonce) המשמש למניעת התקפות שחזור בגודל 64 ביטים (8 בתים = 2 מילים).
3. מונה בלוקים בגודל 64 ביטים (8 בתים = 2 מילים).
4. 4 מילים קבועות (Constant) בגודל 128 ביטים (16 בתים = 4 מילים).

5. טבלה 1 – initial matrix

C0 32 bits (0,0)	K0 32 bits (0,1)	K1 32 bits (0,2)	K2 32 bits (0,3)
K3 32 bits (1,0)	C1 32 bits (1,1)	N0 32 bits (1,2)	N1 32 bits (1,3)
Cnt0 32 bits (2,0)	Cnt1 32 bits (2,1)	C2 32 bits (2,2)	K4 32 bits (2,3)
K5 32 bits (3,0)	K6 32 bits (3,1)	K7 32 bits (3,2)	C3 32 bits (3,3)

על מנת לייצר את הזרם הקריפטוגרפי הפסאודו-אקראי עבור בלוק מסוים (רכיב מונה הבלוקים במטריצה ההתחלתית מייחד את הבלוק המוצפן), נחבר בין המטריצה ההתחלתית לבין תוצאת פונקציית הגיבוב עליה מספר פעמים שנבחר. Salsa20 כלומר המטריצה עברה 10 סבבים של טרנספורמציות שעליהם נסביר בהמשך.

פעולת XOR בין הזרם הקריפטוגרפי (512 bits) לבין הביטים של המידע שרוצים להצפין (Plain text) ייצור את המידע המוצפן (Cipher text).

מכיוון שתהליך ההצפנה הסופי מתבצע ע"י פעולת XOR ניתן להוכיח על סמך משוואות (1-3) שאלגוריתם ההצפנה הינו גם אלגוריתם הפענוח, מכיוון שמדובר באותו זרם קריפטוגרפי (מפתח הצפנה – K) לבלוק ספציפי.

כלומר, לאחר המימוש נקבל מערכת אחת שיודעת לעשות את שתי הפעולות ההפוכות זו מזו, פענוח והצפנה.

$$C = K \text{ xor } P \rightarrow K \text{ xor } C = K \text{ xor } (K \text{ xor } P) \quad (1)$$

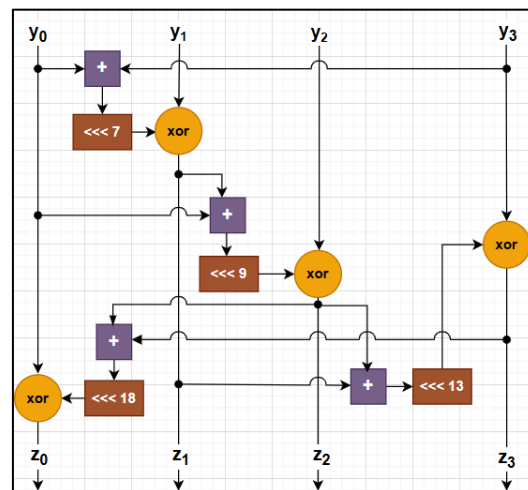
$$K \text{ xor } K = 0 ; 0 \text{ xor } P = P \quad (2)$$

$$\rightarrow K \text{ xor } C = P \quad (3)$$

ישנם גרסאות נגזרות של אלגוריתם זה, כמו ChaCha20 המשמשות כיום בפרוטוקולים כמו TLS ו HTTPS.

ב.2.א. פונקציית Quarter Round

בסיס פונקציית הגיבוב היא פונקציית ה quarter round, פונקציה זו מקבלת קלט של 4 מילים (מילה = 32 ביט) ומוציאה קלט של 4 מילים. באיור ב.1 מוצג תיאור סכמתי של הפונקציה quarter round.



איור ב.1 - תיאור סכמתי של הפונקציה Quarter Round

פונקציית ה quarter round מערבלת את ארבעת מילות הקלט ע"י טרנספורמציות של XOR, חיבור והזזות מעגליות שמאלה (rotate).

כמו כן, ניכר ההבדל במיקום כל מילה בקלט, המילה תעבור טרנספורמציה שונה בכל מיקום, יבוא לידי ביטוי בפונקציות המתקדמות יותר.

ב.2.ב. פונקציית Row Round

פונקציה זו מקבלת כקלט מטריצה 4×4 של 16 מילים, המטריצה בנויה כמתואר בטבלה (2).

טבלה 2 – מטריצת הקלט לפונקציות Row Round ו Column Round

Y0 32 bits (0,0)	Y1 32 bits (0,1)	Y2 32 bits (0,2)	Y3 32 bits (0,3)
Y4 32 bits (1,0)	Y5 32 bits (1,1)	Y6 32 bits (1,2)	Y7 32 bits (1,3)
Y8 32 bits (2,0)	Y9 32 bits (2,1)	Y10 32 bits (2,2)	Y11 32 bits (2,3)
Y12 32 bits (3,0)	Y13 32 bits (3,1)	Y14 32 bits (3,2)	Y15 32 bits (3,3)

הפונקציה מפעילה את פונקציית Quarter Round על כל אחד מארבעת השורות ומחזירה פלט של מטריצה 4×4 בהתאמה, כלומר היציאה של הטרנספורמציה על Y0, הנקראת Z0, תהיה באותו מיקום במטריצת הפלט.

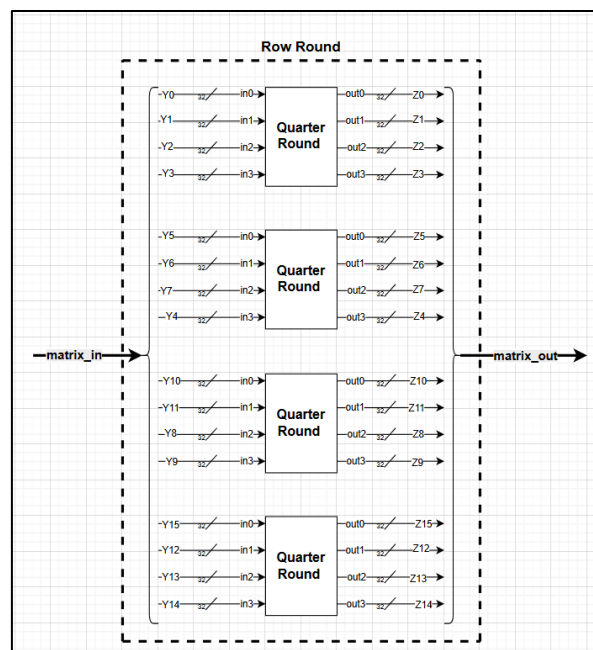
בנוסף, סדר המילים של כל שורה הנכנסים כקלט לפונקציית Quarter Round משתנה בכל שורה ולא נכנס לפי הסדר הסידורי אלא לפי המתואר במשוואות (7)-(4). באיור 2.ב מוצג דיאגרמת הבלוקים של פונקציית Row Round.

$$(Z_0, Z_1, Z_2, Z_3) = \text{QuarterRound}(Y_0, Y_1, Y_2, Y_3) \quad (4)$$

$$(Z_5, Z_6, Z_7, Z_4) = \text{QuarterRound}(Y_5, Y_6, Y_7, Y_4) \quad (5)$$

$$(Z_{10}, Z_{11}, Z_8, Z_9) = \text{QuarterRound}(Y_{10}, Y_{11}, Y_8, Y_9) \quad (6)$$

$$(Z_{15}, Z_{12}, Z_{13}, Z_{14}) = \text{QuarterRound}(Y_{15}, Y_{12}, Y_{13}, Y_{14}) \quad (7)$$



2- איור 2.ב - דיאגרמת הבלוקים של פונקציית Row Round.

ג.2.ב. פונקציית Column Round

פונקציה זו מקבלת כקלט מטריצה 4×4 של 16 מילים, המטריצה בנויה כמתואר בטבלה (2).

הפונקציה מפעילה את פונקציית Quarter Round על כל אחד מארבעת העמודות ומחזירה פלט של מטריצה 4×4 בהתאמה, כלומר היציאה של הטרנספורמציה על Y0, הנקראת Z0, תהיה באותו מיקום במטריצת הפלט.

בנוסף, סדר המילים של כל עמודה הנכנסים כקלט לפונקציית Quarter Round משתנה בכל עמודה ולא

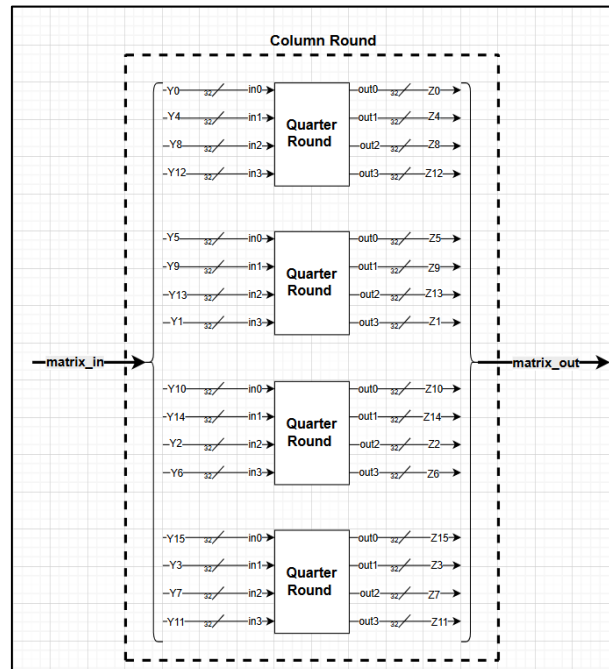
נכנס לפי הסדר הסידורי אלא לפי המתואר במשוואות (8-11). באיור 3. מוצג דיאגרמת הבלוקים של פונקציית Column Round.

$$(Z_0, Z_4, Z_8, Z_{12}) = \text{QuarterRound}(Y_0, Y_4, Y_8, Y_{12}) \quad (8)$$

$$(Z_5, Z_9, Z_{13}, Z_1) = \text{QuarterRound}(Y_5, Y_9, Y_{13}, Y_1) \quad (9)$$

$$(Z_{10}, Z_{14}, Z_2, Z_6) = \text{QuarterRound}(Y_{10}, Y_{14}, Y_2, Y_6) \quad (10)$$

$$(Z_{15}, Z_3, Z_7, Z_{11}) = \text{QuarterRound}(Y_{15}, Y_3, Y_7, Y_{11}) \quad (11)$$

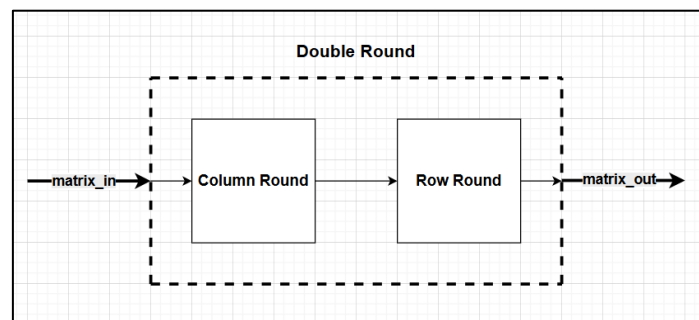


איור 3. - דיאגרמת הבלוקים של פונקציית Column Round.

ב.2.ד. פונקציית Double Round

פונקציה זו בעצם מפעילה על מטריצת הקלט (המטריצה המתוארת בטבלה (2)) את פונקציית העמודות ועל הפלט שמתקבל מפעילה את פונקציית השורות, כמתואר במשוואה (12). כלומר, כל מילה עוברת טרנספורמציה פעמיים, פעם אחת דרך העמודות ופעם שניה דרך השורות. פונקציה זו בעצם מוגדרת כסיבוב אחד של פונקציית הגיבוב. באיור 4. מוצג דיאגרמת הבלוקים של פונקציית Double Round.

$$\text{DoubleRound}(Y_0, \dots, Y_{15}) = \text{RowRound}(\text{ColumnRound}(Y_0, \dots, Y_{15})) \quad (12)$$



איור 4. - דיאגרמת הבלוקים של פונקציית Double Round.

ב.2.ה. פונקציית הגיבוב SALSA20

פונקציה זו מפעילה את פונקציית Double Round על מטריצת הקלט (המטריצה המתוארת בטבלה (2)) 10 פעמים ברציפות ומוסיפה לפלט זה את מטריצת הקלט, כמתואר במשוואה (13). מכיון שכל סיבוב של Double Round בעצם מבצע טרנספורמציה על כל מילה פעמיים הצופן נקרא Salsa20 (10 סיבובים כפול 2 שווה 20).

$$Salsa20(Y_0, \dots, Y_{15}) = (Y_0, \dots, Y_{15}) + DoubleRound^{10}(Y_0, \dots, Y_{15}) \quad (13)$$

יתרה מכך, כאשר מעוניינים בפונקציית הצפנה מהירה יותר על חשבון ביטחון אפשר להפחית במספר הסבבים. האופציות המומלצות הן 8 או 12 סבבים.

ב.3. רקע על ה-FPGAs השונים

בפרויקט זה נעשה שימוש בשני כרטיסי FPGA שונים, מתוצרת שתי חברות שונות. לכל כרטיס ולכל יצרן קיימים כלי פיתוח ושיטות עבודה ייחודיים. לכן, נסקור בקצרה כל אחד מהכרטיסים, תוך התמקדות ברקע כללי, ממשקים ייחודיים ותכונות עיקריות.

ב.3.א. Basys 3 – Xilinx AMD

Basys3 הינו לוח פיתוח FPGA ברמה בסיסית, המיועד ללימוד והוראה של עיצוב דיגיטלי. הלוח מבוסס על ארכיטקטורת ה-FPGA מסוג Artix-7 של AMD (לשעבר Xilinx) ומיועד לעבודה עם סביבת הפיתוח Vivado.

הלוח תומך בחיבורי PMOD, USB, VGA. בנוסף, על הבורד ישנם מתגים, לחצנים, תצוגת SSD וחיבורי GPIO רבים. על גבי הלוח קיים מקור שעות יחיד בתדר של 100MHz. [4]

ב.3.ב. DE10-Lite - Intel FPGA

לוח פיתוח DE10-Lite הינו FPGA חסכוני, המבוסס על רכיב ממשפחת "MAX 10" של חברת הטכנולוגיה Intel. הלוח מספק פלטפורמת חומרה יציבה ונוחה למגוון פרויקטים.

סביבת הפיתוח המשמשת לעבודה עם לוח זה היא Quartus (לשעבר של חברת Altera). הלוח תומך בממשקים כגון VGA וחיבור תואם Arduino UNO R3, וכן כולל זיכרון SDRAM. בנוסף קיימים עגבי הלוח שני מקורות שעות, כל אחד בתדר של 50MHz. [5]

ב.4. פרוטוקול התקשורת UART

מימוש הפרויקט בחומרה על FPGA דורש תקשורת בין המחשב (המשתמש) לבין הבורד עצמו. תקשורת זו מתבצעת על ידי חיבור כבל USB וע"י הגדרת הפינים RX ו TX על הבורד בהתאם לבורד שאיתו עובדים, ניתן למצוא את כל הפרטים בדפי הנתונים של כל בורד.

פרוטוקול התקשורת (Universal Asynchronous Receiver/Transmitter) UART [6], הוא פרוטוקול תקשורת סריאלי אסינכרוני המשמש להעברת נתונים בין התקנים אלקטרוניים, כגון מחשב, מיקרו-בקרים, FPGA ועוד. הפרוטוקול פועל ללא שעות משותף בין השולח למקבל, מה שמקל על היישום שלו אך דורש תיאום בין הצדדים מבחינת קצב השידור (Baud Rate).

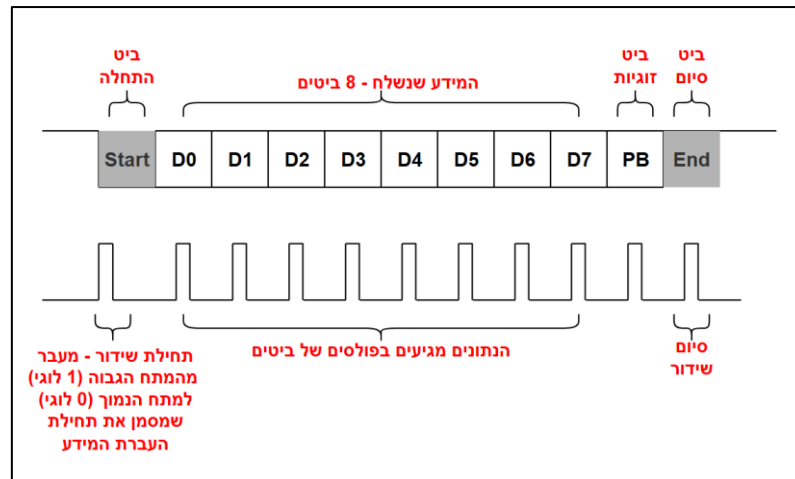
• עקרונות הפרוטוקול:

UART בעמדת השולח ממיר נתונים מקבילים לזרם נתונים סריאלי (ביט-ביט) ואילו בעמדת המקלט ממיר זרם נתונים סריאלי לנתונים מקבילים.

פרוטוקול זה מבוסס על כמה הגדרות פשוטות:

- ביט התחלה – סימון תחילת המסגרת באמצעות מעבר מרמה גבוהה (1) לרמה נמוכה (0).
- ביטים של המידע – הנתונים מועברים ב 5-9 ביטים, 8 ביטים (בית) זה הנפוץ ביותר.
- ביט זוגיות – ביט אופציונלי המשמש לבדיקת שגיאות.
- ביט עצירה – סימון סוף המסגרת ברמה גבוהה – בד"כ ביט אחד אפשר גם שנים.

באור ב.5 מוצג תיאור מבנה חבילת נתונים בתקשורת UART.



5- איור ב.5 - תיאור מבנה חבילת נתונים בתקשורת UART.

- קצב השידור (Baud Rate), נמדד ביחידות של ביטים לשנייה (bps). 9,600 bps הינו ערך נפוץ לקצב שידור. חייב להיות סנכרון של קצב השידור בין הצדדים. הביטוי לחישוב מספר מחזורי השעון (N) למעבר ביט אחד מתואר במשוואה (14).

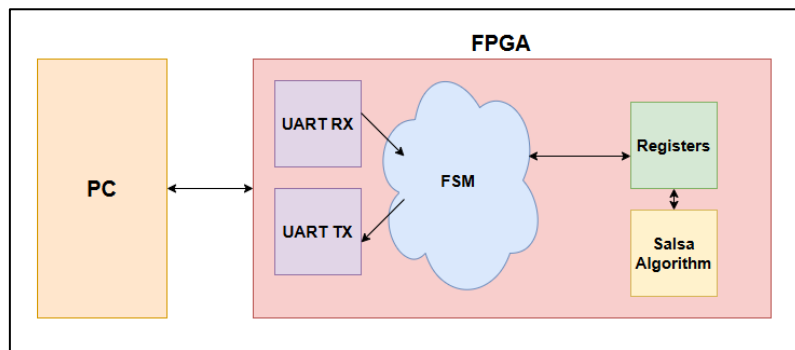
$$N = \frac{f_{[Hz]}}{BaudRate_{[bps]}} \quad (14)$$

ג. פיתוח ושיטות

ג.1. הקדמה לתכנון ואפיון המערכת

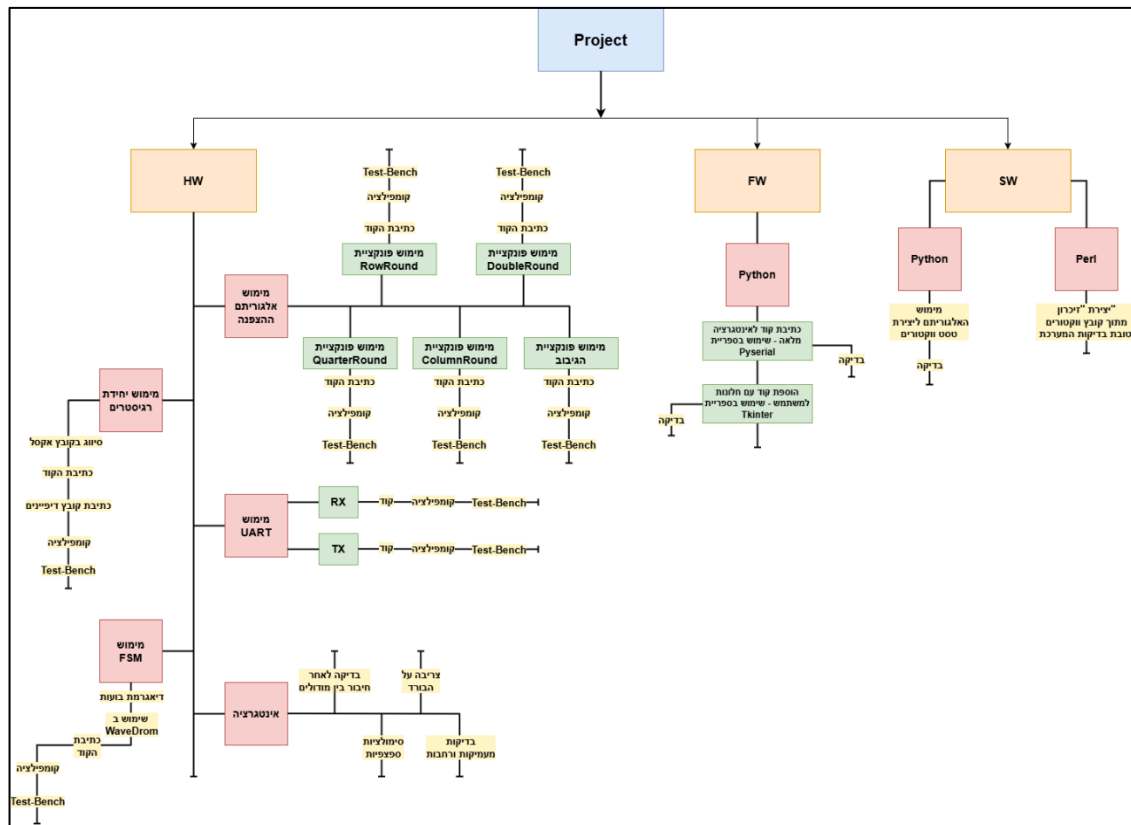
בפרויקט זה נרצה לממש בחומרה אלגוריתם הצפנה על טכנולוגיות FPGA שונות, מימוש חומרתי זה מכיל בתוכו מספר צרכים ואתגרים רחבים על מנת ליצור מימוש חומרתי ברמת גימור איכותית ועניינית. תכנון המערכת השלימה מצריכה מחשבה עמוקה בשיטות כתיבת החומרה ושיטות ארכיטקטורה שונות, לכן באופן ראשוני עלינו לזהות את הצרכים המרכזיים של המערכת ולתכנן שלבי פעולה מוגדרים בהתאם על מנת לעמוד בדרישה.

בנוסף למימוש האלגוריתם, יש לממש את כל האופרציה של המערכת, כולל יצירת אינטגרציה מלאה לשימוש עילי, כפי שמתואר באיור ג.1., כך שהמערכת תקבל נתונים התחלתיים מהמשתמש דרך המחשב ותבצע את הצפנתם תוך התחשבות במגבלות ה-FPGA מבחינת משאבים וזמני תגובה. בנוסף, הפרויקט דורש מימוש חומרה בשפת Verilog.



6- איור ג.1 - דיאגרמת בלוקים עילית של המערכת.

באיור ג.2 ניתן לראות את חלוקת העבודה למשימות ותתי משימות.

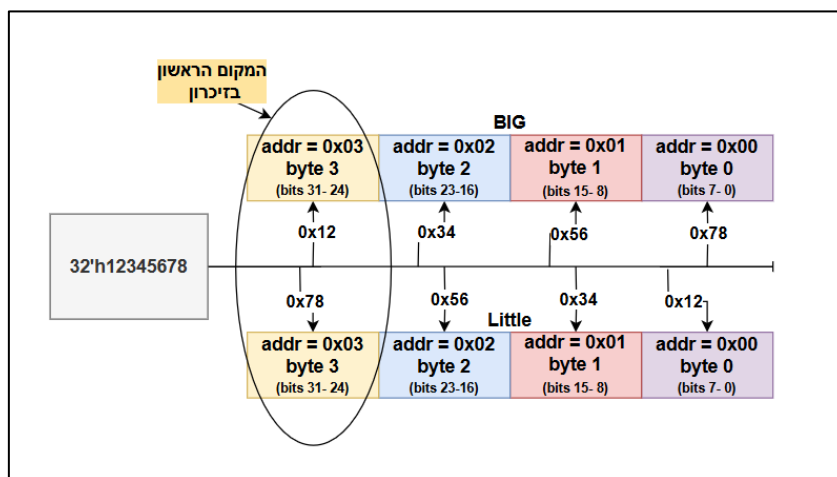


7- איור ג.2 - חלוקת העבודה למשימות

ג.1.א. עקרון ה- Endianness

Endianness [7] (סדר בתיים / סדר הביטים ברמת ה- Byte Order) הוא פרמטר המגדיר את האופן שבו מערכת מחשב או רכיב חומרה מסדרת בתיים בתוך מילים (words), בזיכרון או בתקשורת. פרמטר שקובע כיצד לפרש רצף בתיים כמילים שלמות. ההבחנה העיקרית היא בין שתי שיטות נפוצות כמתואר באיור ג.3. שיטות אלו נקראות:

- Little-Endian - "הבית הקטן" (least significant byte) נשמר ראשון בזיכרון.
- Big-Endian - "הבית הגדול" (most significant byte) נשמר ראשון בזיכרון.



8- איור ג.3 - שיטות ה- Endianness

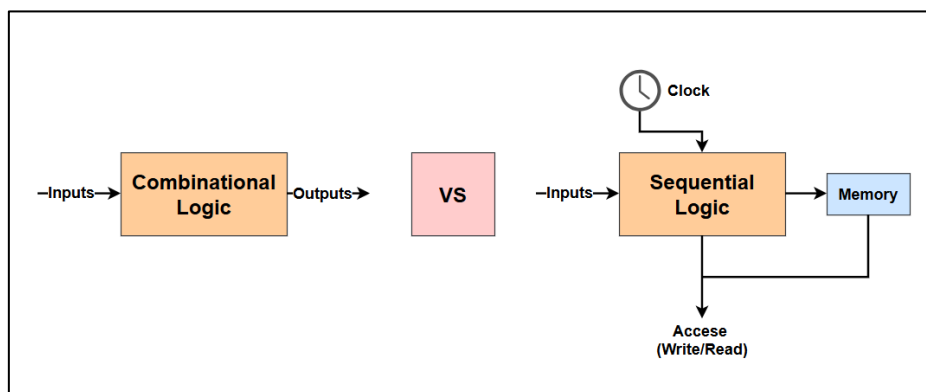
הפרויקט תואם את עקרון ה-Little-Endian (כפי שדורש Salsa20 וכניסות הרגיסטרים הממופות), והדבר משתקף במיפוי הרגיסטרים, בהקפדה על עקביות בין ה-FW ל-HW, ובבדיקות המוודאות השווה נכונה של ערכים על כל שרשרת התקשורת.

ג.1.ב. לוגיקה קומבינטורית וסדרתית

בתכנון מערכות דיגיטליות, קיימת הבחנה עקרונית בין שני סוגי לוגיקה: לוגיקה קומבינטורית ולוגיקה סדרתית [8-9].

הלוגיקה הקומבינטורית מבוססת על פעולות שמבוצעות על קלטים מיידיים בלבד, כאשר כל שינוי בקלט משפיע באופן מידי על הפלט, ללא כל תלות בזמן או במצב קודם. לעומתה, הלוגיקה הסדרתית כוללת רכיבי זיכרון ומבוססת על סנכרון עם שעון – ולכן היא שומרת על מצב פנימי ומאפשרת בניית מערכות מורכבות, כמו מכונות מצבים, רגיסטרים וספריות זיכרון, כמתואר באיור ג.4.

שילוב נכון בין שתי השיטות מאפשר מימוש אפקטיבי של פונקציונליות מתקדמת במעגלים דיגיטליים, והוא מהווה בסיס חשוב לתכנון מערכות חומרה בשפת Verilog וב- FPGA.



9- איור ג.4 - לוגיקה סדרתית VS לוגיקה קומבינטורית

ג.1.1. שיטת Unrolling

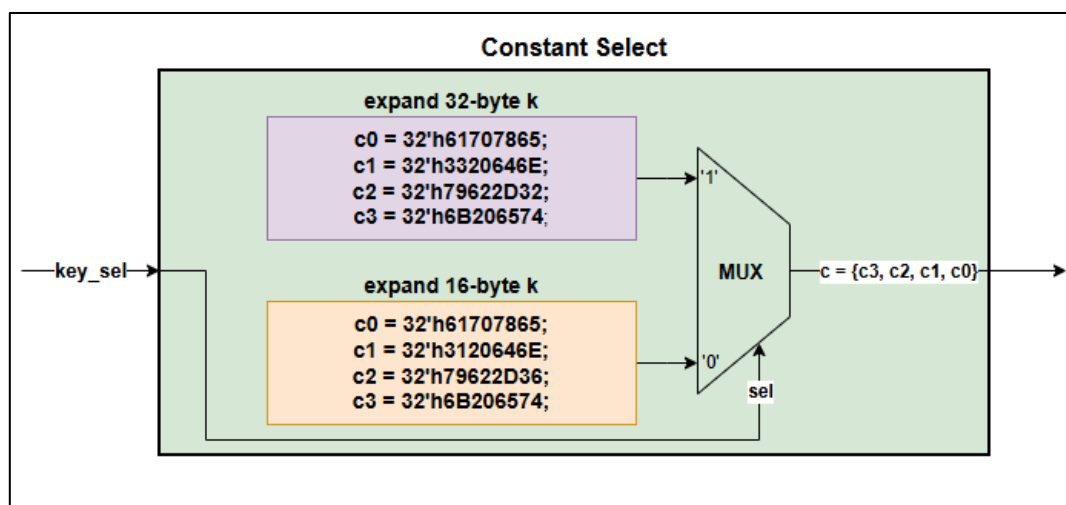
שיטת Unrolling (פריסת לולאה), הינה טכניקה המשמשת בתכנון חומרה לצורך שיפור ביצועים על ידי המרה של לולאות חוזרות למבנה מקבילי. במקום לבצע את פעולות הלולאה באופן סדרתי בכל מחזור שעון, מתבצעת פריסה של הלולאה כך שכל איטרציה ממומשת כחומרה נפרדת הפועלת במקביל. בצורה זו ניתן לצמצם משמעותית את זמן הביצוע של החישוב, במיוחד במעגלים חישוביים או אלגוריתמים שחוזרים על אותה פעולה מספר רב של פעמים.

יתרונות השיטה כוללים שיפור במהירות הפעולה, ניצול יעיל יותר של המקביליות הטבועה ב-FPGA או ב-ASIC, ופשטות בלוגיקת הבקרה. עם זאת, החסרונות העיקריים הם הגדלה משמעותית של צריכת המשאבים (כמו Flip-Flops, LUTs), ולעיתים גם עלייה בצריכת ההספק. לכן, יש לאזן בין דרישות הביצועים לבין מגבלות המשאבים הפיזיים של הרכיב.

ג.2. תכנון החומרה

ג.2.1. constant_select

בלוק הערכים הקבועים נקרא constant_select, ומתואר באיור ג.5.



איור ג.5 - דיאגרמת בלוקים constant_select.

בלוק זה אחראי על קביעת ארבע המילים הקבועות (Constants) הנדרשות לאתחול אלגוריתם ההצפנה Salsa20. המודול מקבל סיגנל קלט ברוחב ביט אחד, המשמש לבחירת גודל המפתח (Key) – כאשר ערך הקלט קובע האם נעשה שימוש במפתח ברוחב 128 ביטים או 256 ביטים. בהתאם לבחירה זו, המודול מייצר את ארבעת המילים הקבועות במבנה ASCII, על פי אחת משתי המחרוזות הידועות:

עבור מפתח ברוחב 256 ביטים – "expand 32-byte k"

עבור מפתח ברוחב 128 ביטים – "expand 16-byte k"

הקבועים נשלפים מתוך המחרוזות לפי עקרון ה-Little Endian, כך שכל מילה נשמרת בזיכרון בסדר בתים הפוך (מהבייט הפחות משמעותי אל המשמעותי).

בטבלה (3) מוצג פירוט הערכים הקבועים והאופן שבו הם נבנים עבור כל אחת מתצורות המפתח.

טבלה 3 – פירוט הערכים הקבועים (Constants) לפי גודל המפתח

שם הקבוע	תפקיד	מפתח – 256 ביטים ("expand 32-byte k")	מפתח – 128 ביטים ("expand 16-byte k")	הערה
C0	מילה קבועה 0	expa = 0x61707865	expa = 0x61707865	זהה
C1	מילה קבועה 1	nd 3 = 0x3320646E	nd 1 = 0x3120646E	שונה
C2	מילה קבועה 2	2-by = 0x79622D32	6-by = 0x79622D36	שונה
C3	מילה קבועה 3	te k = 0x6B206574	te k = 0x6B206574	זהה

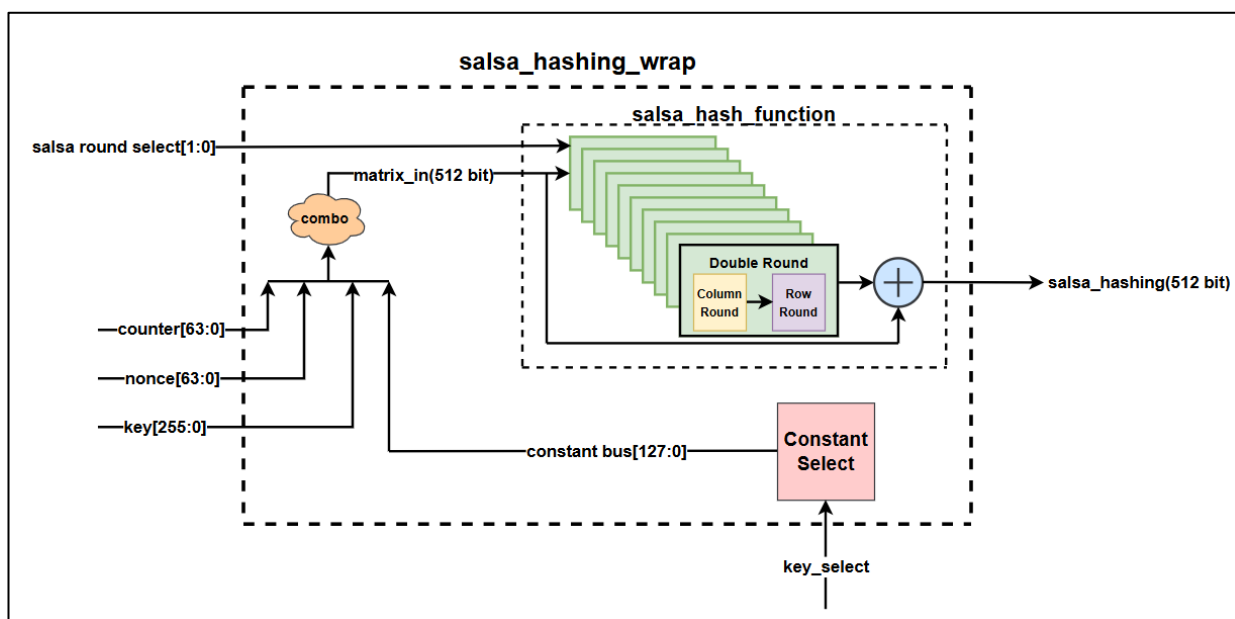
ג.2.ב. תכנון אלגוריתם ההצפנה ג.2.ב.1. שיטת המימוש והמעבר בין Unrolling ללוגיקה סדרתית

בשלב הראשון של הפרויקט בחרתי לממש את הבלוק הלוגי בשיטת Unrolling, במטרה לבחון את היתכנות המימוש ולוודא את תקינות הפונקציונליות של האלגוריתם. בשלב זה המימוש בוצע בצורה קומבינטורית לחלוטין, ללא שימוש בשעון, כך שכל שלבי החישוב בוצעו במקביל ובמחזור שעון יחיד מבחינת התנהגות. גישה זו אפשרה פיתוח מהיר ואימות ראשוני של הפעולה הלוגית.

עם זאת, לאחר ביצוע תהליך Place & Route בכלי Vivado, התברר כי המימוש צורך משאבים רבים מדי ביחס לגודל ה-FPGA שבו נעשה שימוש, ואינו מאפשר סינתזה מלאה של המערכת. בעקבות זאת שיניתי את הגישה וכתבתי את הבלוק באופן סדרתי, תוך שימוש במכונת מצבים (FSM) לניהול השלבים. שינוי זה הפחית משמעותית את השימוש במשאבי הרכיב, תוך שמירה על אותה פונקציונליות בסיסית.

ג.2.ב.2. המימוש בשיטת Unrolling

בלוק ההצפנה בשיטה זו נקרא salsa_hashing_wrap, ומתואר באיור 6.ג. הבלוק בנוי מחיבוריות קומבינטוריות מלאה ומכיל אינסטנס אחד של המודול function. מודול זה מכיל עשרה אינסטנסים של פונקציית ה-double_round, כאשר כל אחת מהן מורכבת מאינסטנס אחד של row_round ואינסטנס נוסף של column_round. כפי שתואר בפרק ב', פונקציות ה-row_round וה-column_round כוללות כל אחת אינסטנס יחיד של פונקציית הבסיס quarter_round, אשר מהווה את יחידת הפעולה הבסיסית בתהליך ההצפנה. בסיום רצף האינסטנסים, מתבצעת פעולת חיבור בין תוצאת הבלוק לבין הקלט המקורי, פעולה זו מהווה את שלב הסיכום הסופי ומגדירה את פונקציית ההצפנה salsa20 במלואה.



11- איור 6.ג - דיאגרמת בלוקים salsa_hashing_wrap.

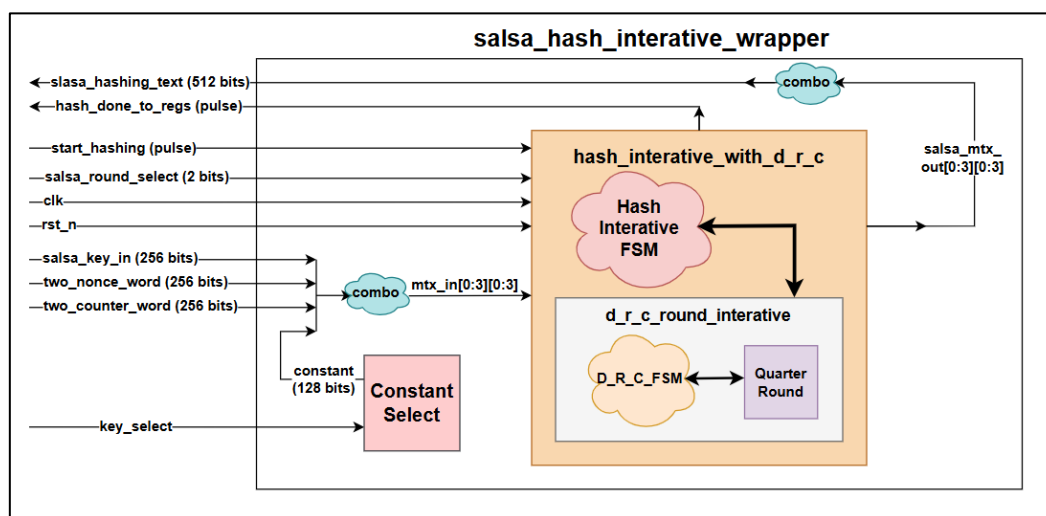
טבלה (4), מציגה את כמות האינסטנסים של כל מודול במימוש בשיטת Unrolling. ניתן לראות כי מספר האינסטנסים הרב בכל רמה יוצר מבנה מקבילי מאוד, אך כתוצאה מכך גם גורם לצריכת שטח (Area) והספק (Power) גבוהה יחסית.

טבלה 4 – מספר אינסטנסים של כל מודול בשיטת Unrolling

Module Name	Total number of instances
quarter_round.sv	20
row_round.sv	10
column_round.sv	10
double_round.sv	10

ג.2.ב.3. המימוש ללא שיטת Unrolling – מימוש סדרתי

באיור ג.7. מוצג ה-top של בלוק ההצפנה שנקרא salsa_hash_interative_wrapper. איור זה מסכם את האינטגרציה בין הבלוקים השונים במערכת וממחיש את אופן יצירת מטריצת הקלט הראשונית עבור האלגוריתם Salsa20.



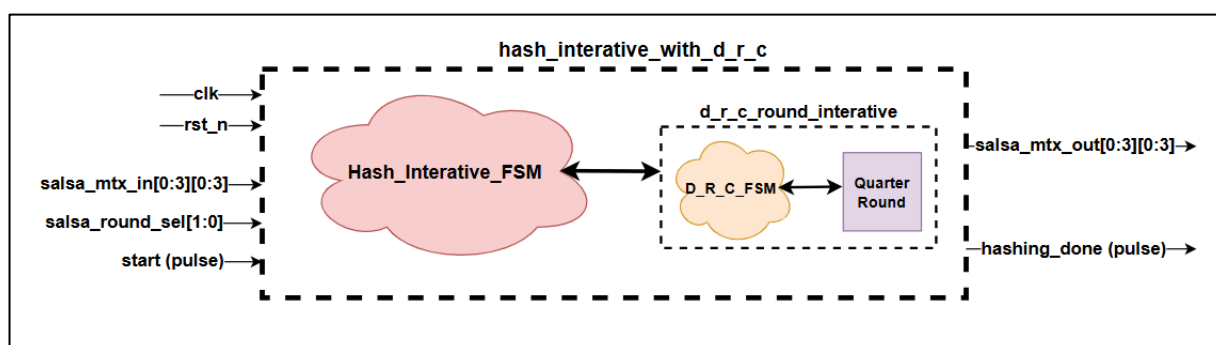
איור ג.7 - דיאגרמת בלוקים salsa_hash_interative_wrapper.

איור זה מבהיר כיצד כל הנתונים מהרגיסטרים ומהבלוקים הקבועים נשזרים ליצירת מטריצת הקלט, שממנה מתחילה המכונה הראשית את פעולתה. כעת נפרט את פעולת ה-FSM הפנימית והאינטראקציה עם מודול ה-d_r_c_round_interactive.

בלוק ההצפנה בשיטה זו נקרא hash_interative_with_d_r_c, ומתואר באיור ג.8. הבלוק מכיל מכונת מצבים ראשית (Hase_Interactive_FSM) ואינסטנס יחיד של המודול d_r_c_round_interactive. מודול זה מכיל בתוכו מכונת מצבים פנימית (d_r_c_FSM) ואינסטנס יחיד של פונקציית ה-quarter_round.

מכונת המצבים מחליפה למעשה את שלושת הפונקציות הקומבינטוריות (column_round, row_round, ו-double_round) ומייצרת פלט שקול לפונקציית double_round עבור מטריצת קלט נתונה.

המכונה הראשית בבלוק זה (Hase_Interactive_FSM) מפעילה את המודול d_r_c_round_interactive מספר פעמים על פי ה-select, ובכל איטרציה מזינה את הפלט של הסבב הקודם כקלט לסבב הבא. בסוף כל האיטרציות, מתבצעת פעולת חיבור בין מטריצת הקלט המקורית לבין הפלט האחרון, ובכך מתקבל הפלט הסופי של פונקציית salsa20.



איור ג.8 - דיאגרמת בלוקים hash_interative_with_d_r_c.

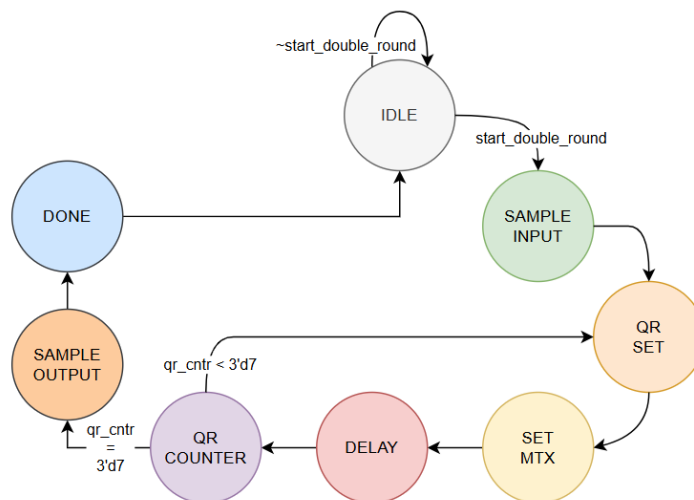
ניתן לראות כי, בניגוד למימוש בשיטת Unrolling, כאן קיים אינסטנס יחיד בלבד של פונקציית הבסיס quarter_round, והמערכת מנצלת את אותו אינסטנס באופן איטרטיבי לאורך כל התהליך. גישה זו מאפשרת חיסכון משמעותי בשטח (Area) ובהספק (Power) על ה-FPGA, מבלי לאבד את הפונקציונליות של אלגוריתם ההצפנה.

על מנת להבין את התנהלות הבלוק הסדרתי ואת תפקודו הפנימי, בהמשך מוצגות דיאגרמות בועות (Bubble Diagrams) עבור כל מכונת מצבים. הדיאגרמות ממחישות את המעברים בין המצבים ואת תזמון

הפעלת המודולים הפנימיים.
במעברים ללא תנאי, המצב משתנה בעליית שעון הבא באופן אוטומטי. במעברים עם תנאי, המכונה נשארת במצב הנוכחי עד שהתרחש האירוע או התנאי הדרוש, ורק אז עוברת למצב הבא.

D R C FSM ○

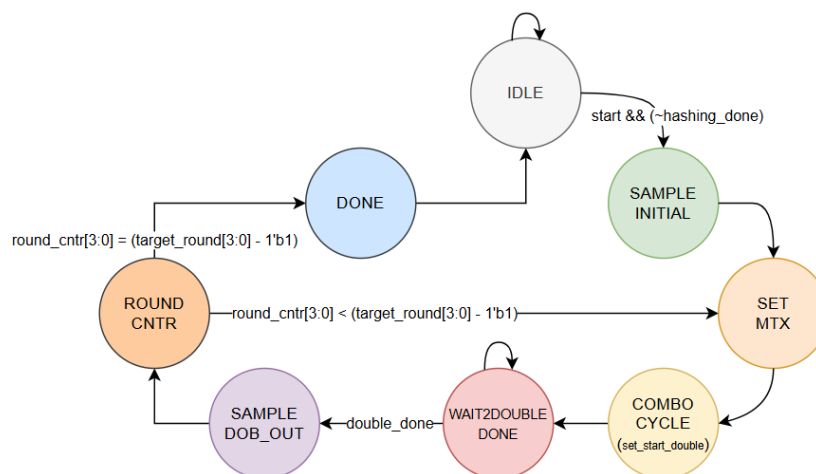
דיאגרמת המצבים, המתוארת באיור ג.9, ממחישה כיצד המכונה הפנימית מחליפה את פונקציות ה- `row_round`, `column_round` ו- `double_round` ומניעה את האינסטנס היחיד של `quarter_round` שמונה פעמים, במטרה ליצור פלט שקול לפונקציית `double_round` (ארבע שורות וארבע עמודות).



14- איור ג.9 - דיאגרמת המצבים של `d_r_c_fsm`.

Hase Iterative FSM ○

דיאגרמת המצבים, המתוארת באיור ג.10, ממחישה כיצד המכונה הראשית מפעילה מספר איטרציות של `double_round` בהתאם לערך ה- `select`, תוך שימוש במודול `d_r_c_round_interative`.



15- איור ג.10 - דיאגרמת המצבים של `hash_iterative_fsm`.

ג.2.ג. תכנון יחידת הרגיסטרים

ג.2.ג.1. רקע והגדרות

רגיסטר (Register) הוא מבנה זיכרון קטן ומהיר הממוקם בתוך רכיבי החומרה. רגיסטרים משמשים לאחסון נתונים בזמן ביצוע פעולות, והגישה אליהם מהירה יותר בהשוואה לזיכרון ראשי או סוגי זיכרונות אחרים, מה שמעניק יתרון משמעותי בתהליכי עיבוד נתונים. רגיסטרים משמשים לשמירת נתוני ביניים, כגון ערכי משתנים, תוצאות חישוביות, כתובות או פקודות, וכן לביצוע פעולות שונות כמו חישובים, השוואות והעברות נתונים.

סוגי הרגיסטרים בפרויקט זה:

- **רגיסטר כללי (General Register):** שומר על ערכי ביניים ללא משמעות מיוחדת.
- **רגיסטר סטטוס (Status Register):** מעיד על מצב של סיגנל מסוים. הסיגנל יכול להיות סיגנל רגיל או מיוחד, כגון סיגנל sticky, אשר נשאר בערכו עד שכתובה אליו תגרום לו לאיפוס (clear) – הסבר מפורט בהמשך.
- **רגיסטר קונטרול (Control Register):** מכיל סיגנלים בעלי משמעות, אשר גורמים לרכיבים אחרים במערכת לפעול או לחכות בהתאם.

בנוסף לכך, קיימים סוגי גישה שונים לרגיסטרים, כפי שמוצג בטבלה (5).

טבלה 5 – תיאור גישות לרגיסטרים

Attribute	Description
RW - Read Write	With FW access, you can both write and read.
RO/V - Read Only / V	In FW access, it is only readable. But it can be changed in hardware.
RW/1C/V - Read Write / 1 Clear / V	In FW access, you can write and read. In addition, when writing 1, the value gets clear. And can be changed in hardware.
RSV - Reserved	Reserved.

ג.2.ג.2. מנגנוני בקרה והתראה – Pulse & Sticky Signals

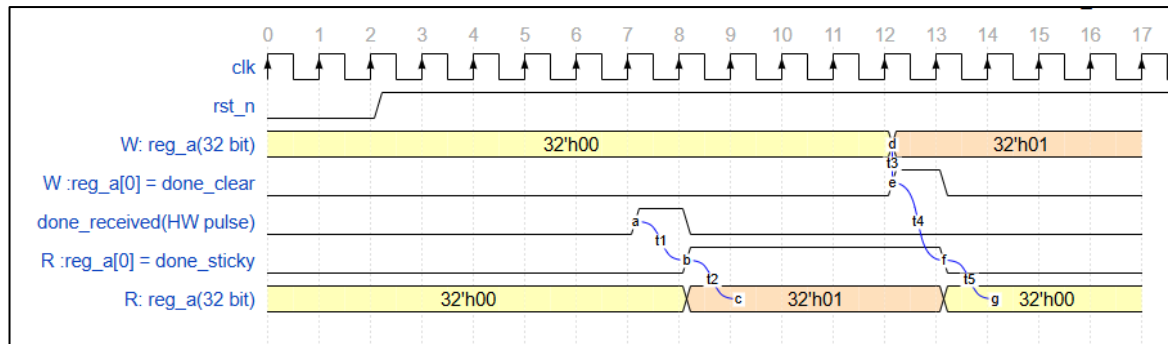
באיר. ג.11. מוצגת דיאגרמת זמנים המתארת את פעולתם של שני סוגי סיגנלים עיקריים במערכת - pulse ו-sticky.

סיגנל **Pulse**, הוא אות קצר בזמן, אשר עולה לרמה לוגית '1' למשך מחזור שעון יחיד בלבד. הפולס יכול להיווצר ממספר גורמים, כגון כתיבה לרגיסטר על ידי Firmware או אירוע חומרתי פנימי במערכת. עם סיום מחזור שעון, הסיגנל חוזר אוטומטית לרמה לוגית '0', ובכך מסמן אירוע חד פעמי (Trigger) מבלי להישאר פעיל לאורך זמן.

סיגנל **Sticky**, לעומת זאת, שומר על ערכו גם לאחר מחזור שעון שבו הופעל. כאשר הוא עולה לרמה '1', הוא נשאר במצב זה עד אשר מתבצעת כתיבה מפורשת של ערך '1' לרגיסטר המתאים לצורך ניקוי (clear) הסיגנל.

במילים אחרות, פעולת ה-clear מתבצעת באמצעות פולס ייעודי, והקריאה לרגיסטר משקפת את מצב ה-sticky בזמן אמת.

גישה זו מאפשרת מעקב אחר מצבים מערכתיים חשובים (כגון סיום תהליך, שגיאה או דגל התראה) גם לאחר שהאירוע כבר התרחש, עד אשר התוכנה בוחרת לאפס אותו באופן יזום.



16- איור ג.11 - דוגמת דיאגרמת זמנים של סיגנלי Pulse ו-Sticky במערכת.

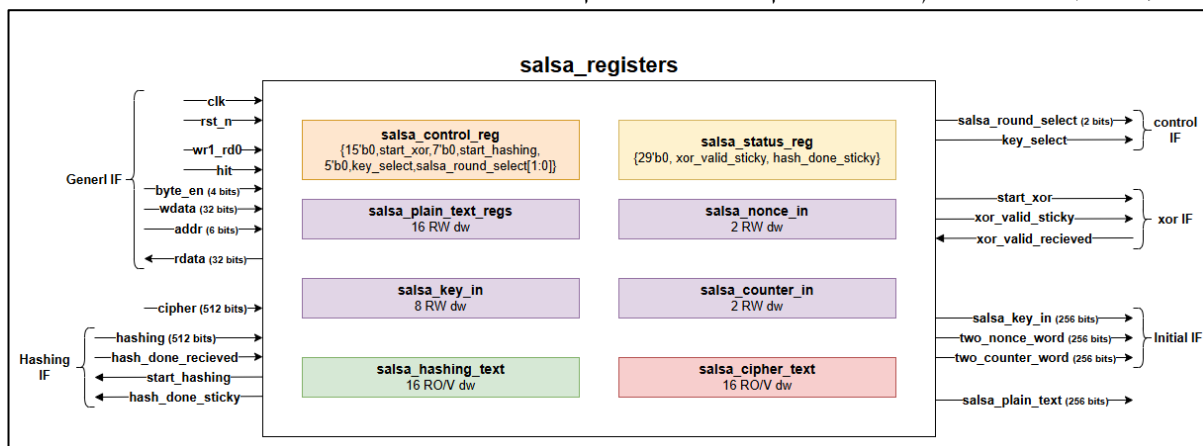
על מנת להמחיש את רצף הפעולות והקשרים בין כתיבת הרגיסטר, קריאת הרגיסטר, יצירת הפולסים, והתנהגות ה-sticky, בטבלה (6) מוצג תיאור מפורט של שלבי התהליך כפי שמודגם באיור ג.11.

טבלה 6 – תיאור שלבי הפעולה של סיגנלי Pulse ו-Sticky במערכת

פעולה במערכת	T
אירוע חומרתי גורם ל- done_recived לעלות לרמה '1'	a
גורם ל sticky לעלות לרמה '1'. done_recived מתאפס כי הוא pulse.	b
Sticky ברמה '1' לכן כאשר נקרא את הרגיסטר נראה '1' במיקום המתאים.	b
בקריאת הרגיסטר נקבל '1' כמצופה.	c
כותבים לרגיסטר את הערך '1' למיקום של done_clear.	d
done_clear עולה לרמה '1' למשך מחזור שעות – כי הוא pulse.	e
done_clear ברמה '1', לכן נצפה לראות את done_sticky מתאפס.	e
done_sticky מתאפס כמצופה.	f
done_sticky יורד לרמה '0' לכן כאשר נקרא את הרגיסטר נצפה לקבל '0' במיקום המתאים.	f
בקריאת הרגיסטר נקבל '0' כמצופה.	g

3.2.2.3. Salsa_Registers

בלוק הרגיסטרים נקרא salsa_registers.sv, ומתואר באיור ג.12. בלוק זה מקבל את הנתונים ההתחלתיים ואת המידע הנדרש לתהליך הצפנה / פיענוח, וכן על אחסון תוצאות הביניים והנתונים הסופיים של ה-hash וה-cipher במהלך פעולת המערכת. הבלוק משמש כממשק תקשורת בין המעבד (ה-Firmware) לבין ליבת ההצפנה, ומאפשר כתיבה וקריאה של נתונים דרך רגיסטרים ייעודיים. באמצעות קריאת הרגיסטרים לאחר סיום תהלי ההצפנה, המשתמש יכול לגשת אל המידע המוצפן ולבצע אימות או בדיקת תקינות. בנוסף, קיימים שני סיגנלים מסוג sticky, העוברים בהיררכיה עד לרמת ה-top, ומשמשים להדלקת נורות חיווי (LEDs) על גבי הבורד, במטרה לספק למשתמש אינדיקציה ויזואלית על מצב המערכת.



17- איור ג.12 - דיאגרמת בלוקים של salsa_registers.

ג.2.ד. קובץ ההגדרות Salsa_Defines

קובץ הגדרות המשמש לריכוז כל ההגדרות הקבועות והמאקרוים הדרושים עבור כלל המודולים בפרויקט. מטרת הקובץ היא ליצור אחידות, קריאות ותחזוקה קלה של הקוד, על-ידי ריכוז כל הפרמטרים הגלובליים במקום אחד, במקום לפזרם לאורך הקבצים השונים.

הקובץ כולל:

- **הגדרות כתובות הרגיסטרים (Register Addresses):**
כתובת ייחודית לכל רגיסטר במערכת, המשמשת את בלוק הרגיסטרים ואת רכיבי ה-Firmware לצורך קריאה וכתובה של נתונים.
- **הגדרות תקשורת Uart:**
מאקרוים המגדירים את קצב השידור ע"פ הגדרות של שעונים ותדירויות.
- **הגדרות מערכת כלליות (System Parameters):**
פרמטרים קבועים כמו גודל כתובת, ערך קבוע לשחרור מכונת המצבים, קבועים, ערכים להגדרת פונקציית quarter_round.

ג.2.ה. תכנון UART

מימוש ה-Uart כולל מודול מקלט (salsa_rx) ומודול משדר (salsa_uart_tx), שתפקודם מבוסס על מדידת מחזורי שעות בהתאם לקבוע BIT_RATE (שמגודר בקובץ define), זיהוי start bit, העברה/קבלה של 8 ביטי נתונים ו-stop bit.

מנגנון הסנכרון (Synchronizer):

מאחר ש-RS-232 הוא אות אסינכרוני, לא מתואם לשעון המערכת, קיימת סכנה למטסטביליות, כלומר לאיבוד מידע וחוסר יכולת להבין האם המידע הוא ברמה גבוהה או נמוכה. לכן, במימוש ה-Uart, קיים שרשרת של שלושה רגיסטרים, (rs232[2:0]), שתפקידם הוא לסנכרן את האות הנכנס עם השעון הפנימי של המערכת. השרשרת הזו מבטיח שהאות ייקלט בצורה יציבה ולא יגרום לבעיות בתזמון הפנימי של ה-FPGA.

ממשק ה-UART במימוש זה מבוסס על קוד קיים מ-GitHub, אשר שימש כנקודת התחלה בלבד על מנת לחסוך זמן פיתוח ולאפשר התמקדות במימוש האלגוריתם המרכזי ברמת החומרה. הקוד עבר ניתוח ובדיקות יסודיות, ובוצעו התאמות במבנה המודולים, ברמת האותות והסינכרוניזציה, כדי להתאים אותו לארכיטקטורה הכללית של מערכת ה-FPGA שבנינו. כמו כן, שופרו תכונות כמו ניהול תקשורת רציפה, טיפול בוקטורי בדיקה בזמן אמת וחיבור למעטפת הבדיקה ב-Python/Perl, על מנת להבטיח תפקוד יציב ואמין בסביבת בדיקה ויישום ממשי.

ג.2.ו. תכנון מכונת המצבים Salsa_FSM

לצורך ניהול הקישוריות בין רכיבי ה-UART לבין יחידת הרגיסטרים ויחידת ההצפנה, נדרש מנגנון בקרה חומרתי אשר יתאם את תהליכי הקריאה והכתיבה ויבצע חיווט פנימי מדויק בין כל הרכיבים במערכת. מכונת מצבים (FSM – Finite State Machine) מהווה שיטה יעילה ומסודרת לניהול תהליכים סדרתיים מורכבים במערכות חומרה.

שלבי תכנון המכונה:

1. **הבנת דרישות המערכת** – זיהוי המצבים הנדרשים, סיגנלי הבקרה והקשרים ביניהם.
2. **תכנון דיאגרמת בועות** – המחשת המעברים בין המצבים והגדרת תנאי המעבר.
3. **המחשת גלים באמצעות WaveDrom Editor** – לבחינת רצף הפעולות, איתור תלויות תזמון והסקת מסקנות תכנוניות.
4. **כתיבת קוד ה-RTL** – מימוש לוגיקת ה-FSM בהתאם לדרישות, תוך הקפדה על שימוש נכון בסיגנלים כגון flag, pulse, ו-sticky signals.

ג.2.ו.1 דרישות המערכת והגדרת המכונה

במערכת קיימים שני מצבים:

- מצב כתיבה (Write Mode)
- מצב קריאה (Read Mode)

שני המצבים הללו משקפים את כיווני התקשורת בין המחשב (PC) לבין ה-FPGA, ומוגדרים כחלק מפרוטוקול התקשורת של המערכת. מטרת מכונת המצבים היא לשלוט באופן רציף על רצף הפעולות הנדרש בתהליך הקריאה והכתיבה, בהתאם לפרוטוקול. המעברים בין המצבים נקבעים על פי אותות תקשורת נכנסים (כמו סיום קבלת בית, אישור פקודה, או אות busy) ועל פי תנאי סיום פנימיים (כמו סיום כתיבה לרגיסטרים או שליחת הנתונים למחשב).

פירוט המצבים:

1. מצב כתיבה (Write):

במצב זה מתבצעת כתיבה אל ה-FPGA מהמחשב.

רצף המצבים כולל:

- יציאה ממצב Idle עם קבלת הודעה ראשונית.
- קבלת כתובת רגיסטר יעד.
- קבלת פקודת כתיבה.
- קבלת הנתונים לכתיבה דרך ממשק ה-RX.
- הפעלת פקודות לכתיבה לרגיסטרים הפנימיים.

2. מצב קריאה (Read):

במצב זה מתבצעת קריאה מה-FPGA אל המחשב.

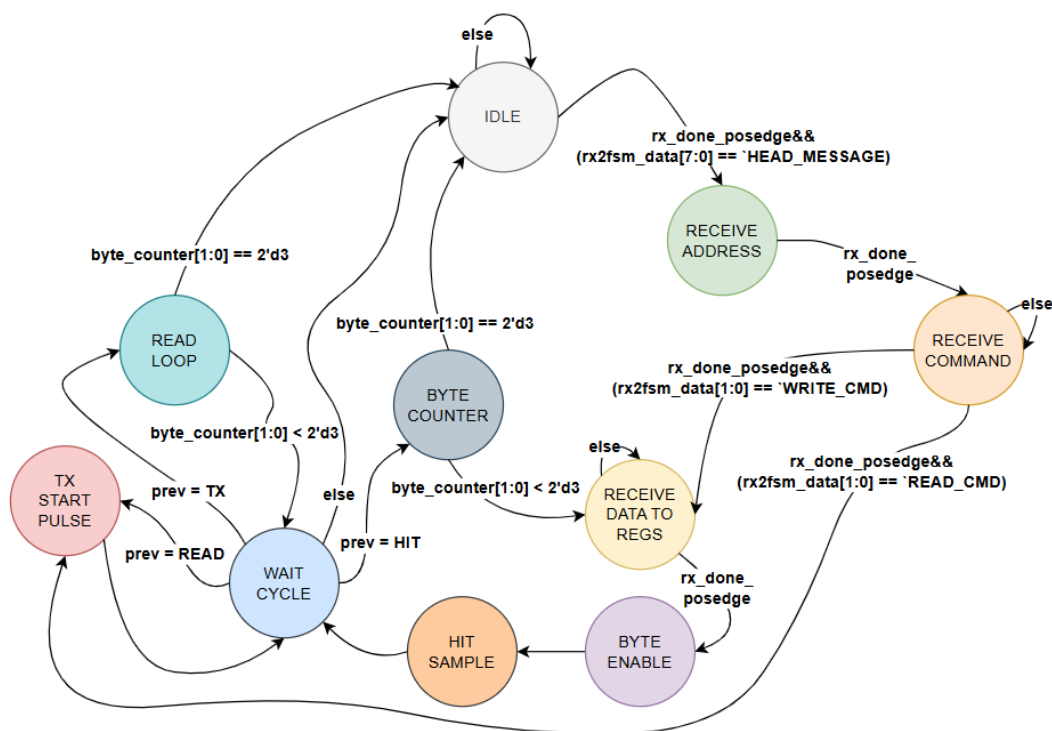
רצף המצבים כולל:

- יציאה ממצב Idle עם קבלת הודעה ראשונית.
- קבלת כתובת רגיסטר מקור.
- קבלת פקודת קריאה.
- שליפת הנתונים מהרגיסטרים הפנימיים.
- שליחת הנתונים דרך ממשק ה-TX למחשב.

ניתן לראות כי בשני מצבי הפעולה קיימים שלבים זהים — כגון קבלת כתובת, קבלת פקודה וניהול תקשורת ראשונית. לכן, בחרתי לממש מכונת מצבים אחת המטפלת הן בקריאות והן בכתיבות. גישה זו חוסכת לוגיקה מיותרת, מפשטת את התחזוקה ומאפשרת סנכרון קל וברור יותר בין רכיבי התקשורת למערכת הרגיסטרים וההצפנה.

ג.2.1.2 דיאגרמת הבעות

באיור ג.13. מוצגת דיאגרמת הבעות של מכונת המצבים Salsa_FSM.



איור ג.13 - דיאגרמת הבעות של מכונת המצבים Salsa_FSM.

מאחר ודיאגרמת הבעות של מכונת המצבים (איור ג.13) הינה מורכבת ומכילה מספר רב של מצבים ומעברים, בטבלה (7) מוצגת סקירה מסכמת של המצבים המרכזיים, התנאים למעבר ביניהם והפעולות המתבצעות בכל אחד מהם.

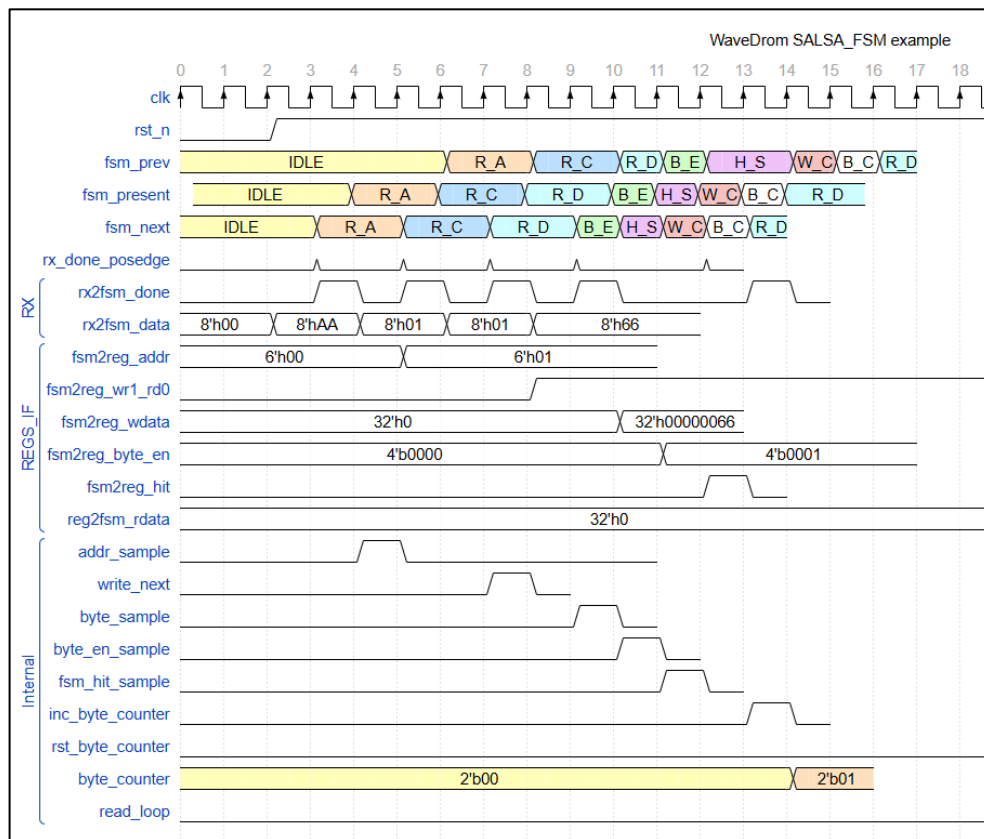
טבלה 7 – פירוט המצבים במכונה

שם המצב	תיאור	תנאי מעבר	פעולה עיקרית
IDLE	ממתין לפקודת שחרור	Rx_done & HEAD_MSG	אתחול
RECEIVE_ADDRESS	מקבל כתובת – דוגם אותה	Rx_done	אתחול כתובת
RECEIVE_COMMAND	מקבל פקודה	על פי הפקודה – כתיבה/קריאה ו Rx_done	אתחול פקודה
RECEIVE_DATA	מקבל בית	Rx_done	לולאת כתיבת הבתים- בית
BYTE_ENABLE	דוגם בית	רציף	אחר בית
HIT_SAMPLE	מאתחל hit לרגיסטרים	רציף	דיליי
WAIT_CYCLE	דיליי של מחזור שעות בין מצבים קריטיים	המצב הקודם קובע לאן ללכת במצב הבא.	
BYTE_COUNTER	משתמש במונה לטובת הכתיבה.	ערך המונה- קטן מ-3 ממשיך לעוד איטרציה, אחרת idle	מונה
TX_START	מאתחל את סיגנל TX לרכיב ה-TX	רציף	אתחול start
READ_LOOP	משתמש במונה לטובת קריאה בית בית מה dw שקיבלנו מהרגיסטר ושולח ל- PC ע"י ה-TX	tx2fsm_done	קריאה מה-FPGA למחשב (PC)

WaveDrom Editor 3.1.2.1

לצורך המחשה וניתוח התנהגות מכונת המצבים בזמן אמת, נעשה שימוש בכלי WaveDrom – כלי גרפי המאפשר תיאור ויזואלי של גלי אותות (waveforms) באמצעות קוד טקסטואלי פשוט. הכלי מאפשר להציג בצורה ברורה את רצף האותות, התזמונים ביניהם והקשרים הלוגיים בין סיגנלים שונים, ובכך מהווה כלי עזר יעיל בתכנון ובאימות מערכות דיגיטליות.

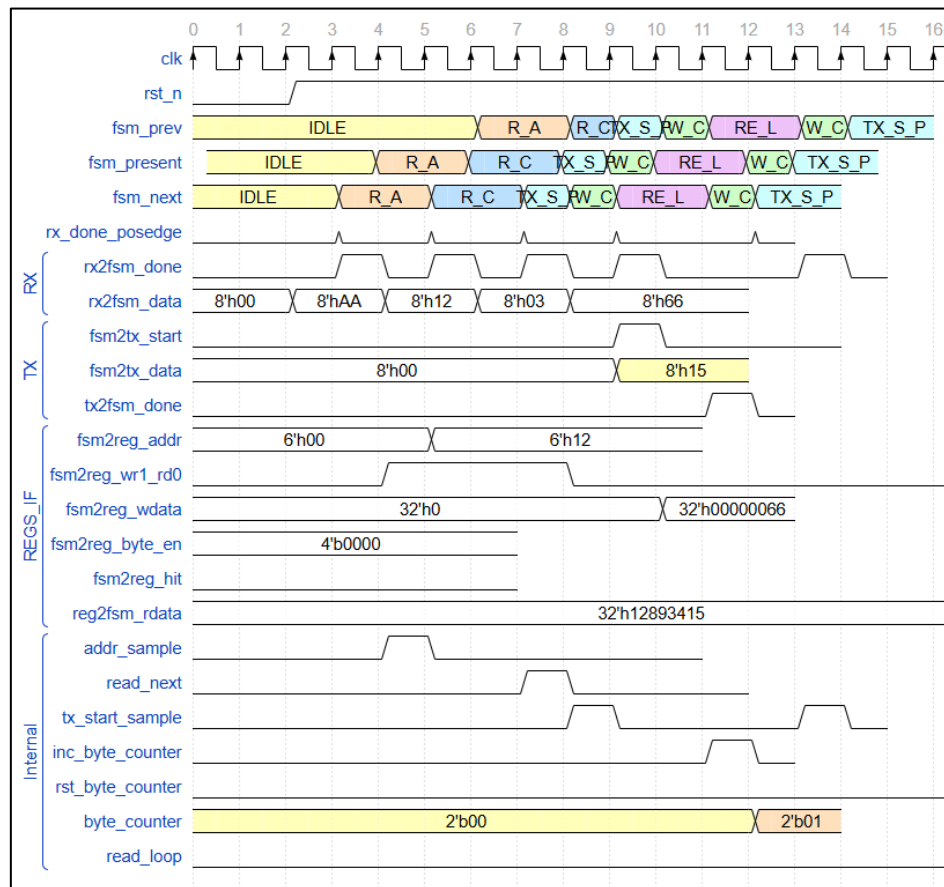
באמצעות הכלי תוכננה דוגמת זמנים עבור מכונת המצבים, המדגימה את התנהגות הסיגנלים המרכזיים לאורך זמן ואת המעברים בין המצבים. באיור ג.14. מוצגת דוגמת גלים (waveform) המתארת את רצף הפעולות של מכונת המצבים salsa_FSM, סיבוב יחיד מתוך ארבעה בקבלת המידע מהמחשב (מצב RX).



איור ג.14 - דוגמת גלים חלקית של מכונת המצבים salsa_fsm במצב RX.

נדגיש שבמציאות סיגנל ה-rx_done לא מסתיים לאחר מחזור שעון יחיד אלא לפי קצב ה-baud rate. על מנת ליצור דוגמה עשיתי כאילו מסתיים לאחר מחזור שעון יחיד.

כמו כן, מוצגת באיור ג.15 דוגמת גלים חלקית של מכוונת המצבים במצב TX.



20- איור ג.15 - דוגמת גלים חלקית של מכוונת המצבים `salsa_fsm` במצב TX.

ג.3. תכנון Salsa_top והגדרות constraints

ג.3.א. Basys3 - salsa_top

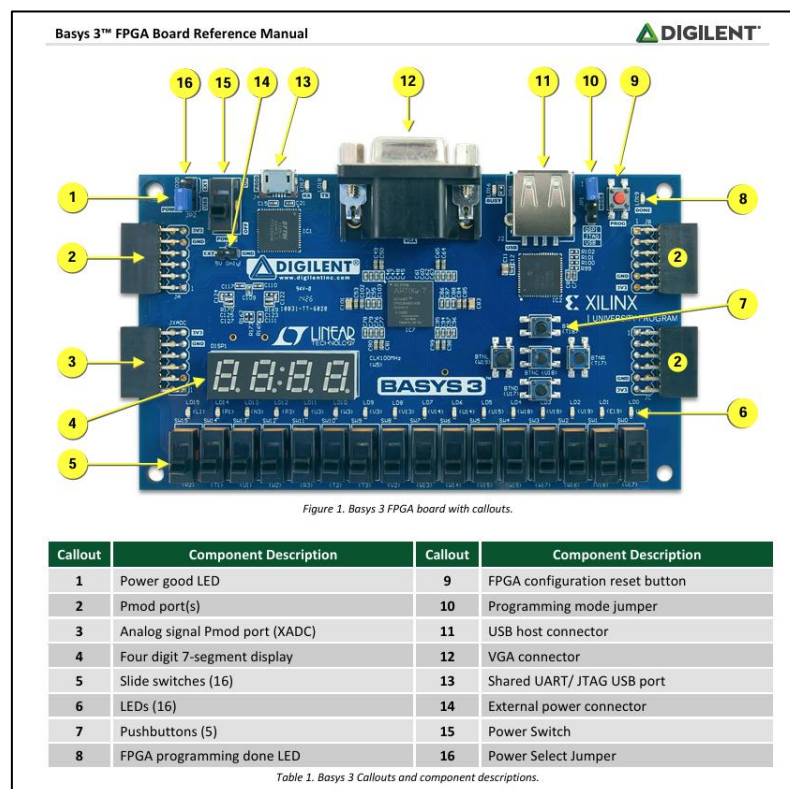
בטבלה (8) מוצגים סיגנלי הכניסה והיציאה מהמודול הראשי : top_salsa_xilinx, בנוסף ניתן לראות בטבלה את מיקום הסיגנלים על הבורד.

בנספח ג' מצורף קובץ ה- xdc שמגדיר הגדרות בסיסיות, שעונים, ריסטים ומיקום סיגנלים על הבורד.

טבלה 8 – פירוט כניסות ויציאות salsa_top.

Signal Name	Width	I/O	Description	Pin Location
salsa_top_100mhz_clk	1	Input	Basys3 clk	W5
salsa_top_rst_n	1	Input	rst_n – active low	R2 -> switch 15
rs232_rx_top	1	Input	Rx input bit	B18
rs_232_tx_top	1	Output	Tx output bit	A18
hash_done_sticky_top	1	Output	Hashing done sticky	U16 -> Led 0
xor_valid_sticky_top	1	Output	Xor is valid sticky	E19 -> Led 1
salsa_top_pll_locked	1	Output	PLL locked indication	U19 -> Led 2
anode_set_top	4	Output	SSD anode set	U2, U4, V2, W4
dp_set_top	1	Output	SSD dp set	V7
seven_seg_top	7	Output	SSD	W7, W6, U8, V8, U5, V5, U7

באיור ג.16. ניתן לראות את האופציות המרובות הקיימות בבורד, כמו שניתן לראות בטבלה (8) השתמשתי בחלק לא גדול.

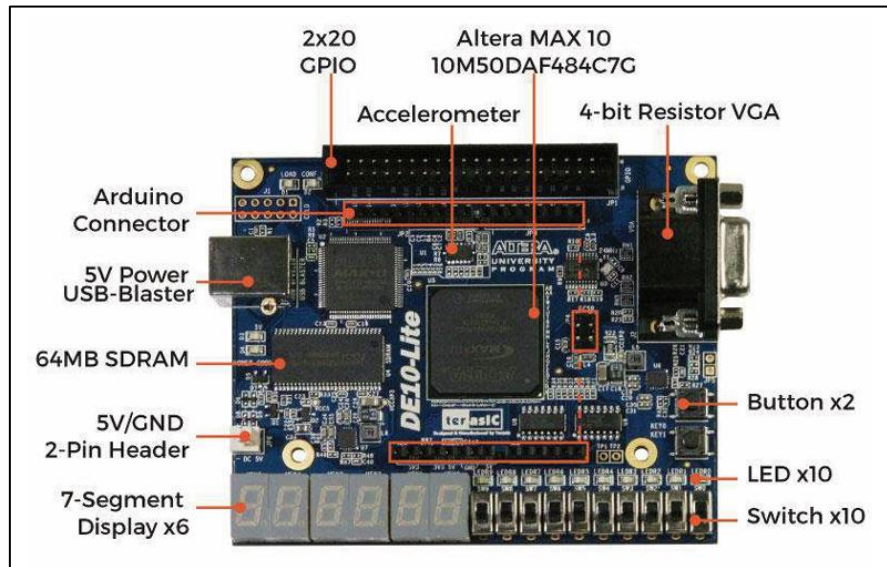


21- איור ג.16 - Basys3 component description מתוך דפי הנתונים.

הסיגנל, salsa_top_pll_locked, מהווה output מה- PLL IP ונותן אינדיקציה כאשר השעון החדש יציב. אני חוברתי את הסיגנל הזה ל- led על הבורד שאוכל לראות באופן ויזואלי את היציבות ורק אז להמשיך ב- flow.

ג.3.ב. DE10-Lite - salsa_top

באיור ג.17 ניתן לראות את הבורד בהסתכלות עילית.



22- איור ג.17 - DE10-Lite Layout and components מתוך דפי הנתונים.

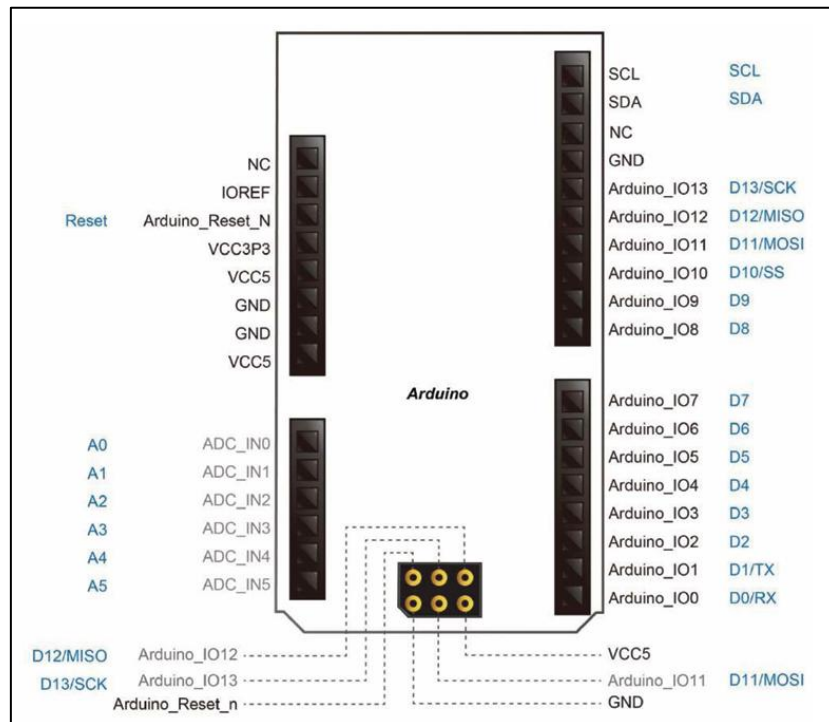
בטבלה (9) מוצגים סיגנלי הכניסה והיציאה מהמודול הראשי : `top_salsa_intel`, בנוסף ניתן לראות בטבלה את מיקום הסיגנלים על הבורד.

בנספח ד' מצורף קובץ ה- `sdc` שמגדיר שעונים, ריסטים ו- `I/O`. כמו כן, בנספח ה' מצורף קובץ ה- `qsf`, הקובץ שמכיל את מיקומי הסיגנלים על הבורד.

טבלה 9 – פירוט כניסות ויציאות `salsa_top`.

Signal Name	Width	I/O	Description	Pin Location
<code>salsa_top_50mhz_clk</code>	1	Input	Basys3 clk	PIN_P11
<code>salsa_top_rst_n</code>	1	Input	<code>rst_n</code> – active low	PIN_F15
<code>rs232_rx_top</code>	1	Input	Rx input bit - Arduino IO0	PIN_AB5 -
<code>rs_232_tx_top</code>	1	Output	Tx output bit - Arduino IO1	PIN_AB6
<code>hash_done_sticky_top</code>	1	Output	Hashing done sticky	PIN_A8
<code>xor_valid_sticky_top</code>	1	Output	Xor is valid sticky	PIN_A9
<code>dp_set_top</code>	1	Output	SSD dp set	PIN_L19
<code>seven_seg_top</code>	7	Output	SSD	PIN_J20,K20, L18, N18, M20, N19, N20

באיור ג.18 ניתן לראות את ה-Arduino הפנימי שיושב על הבורד, כפי שציינתי בטבלה לעיל, השתמשתי בחיבורי ה-RX וה-TX של ה-Arduino על מנת לתקשר עם ה-FPGA.

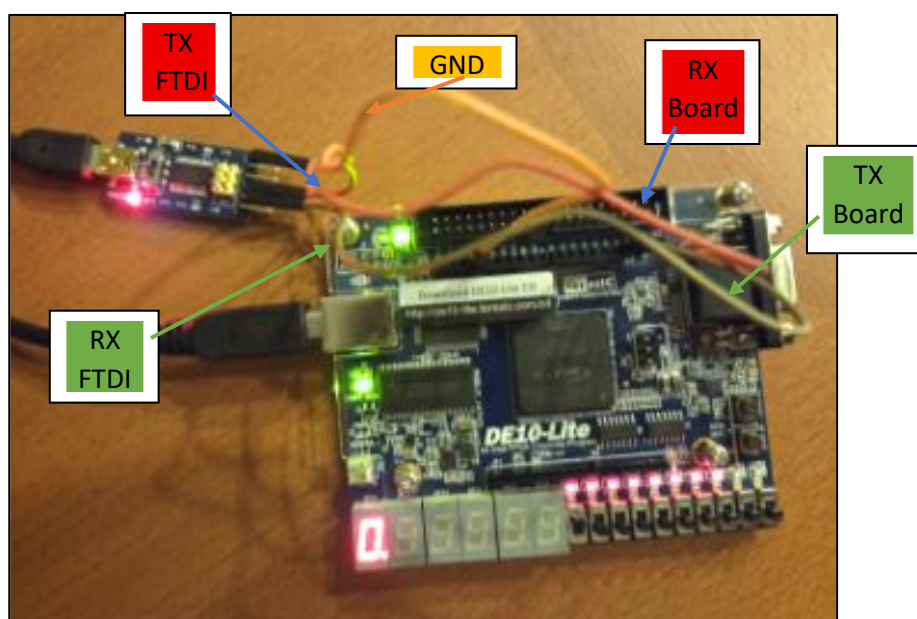


23- איור ג.18 - Arduino UNO3 description מתוך דפי הנתונים.

בשונה מ-Basys3, לצורך מימוש התקשורת בין המערכת למחשב, שולב רכיב FTDI חיצוני, ששימש כגשר בין ממשק ה-USB לבין ממשק ה-UART. כמתואר באיור ג.19, החיבוריות בין ה-FTDI החיצוני לבין הבורד.

החיבוריות עוברת הצלבה בין המחשב לבורד, ה-receiver של הבורד הוא ה-transmitter של המחשב ולהיפך, ה-transmitter של הבורד הוא ה-receiver של המחשב.

24- איור ג.19 - חיבוריות בין FTDI לבורד.



4.4. Python Scripts

שפת Python היא שפה דינמית, מודרנית ורב-תכליתית, הנפוצה במיוחד בתחומי פיתוח אלגוריתמים, בדיקות, סימולציה ו-verification. לפייתון תחביר פשוט וקריא, ספריות מתמטיות עשירות (כגון NumPy) ותמיכה טבעית בעבודה עם מבנים בינאריים, מחרוזות וייצוגי hex מה שהופך אותה לכלי אידיאלי למימוש מוקדם של אלגוריתמים קריפטוגרפיים וליצירת וקטורי בדיקה למערכות חומרה.

יתרונה המרכזי של השפה הוא קיצור משמעותי של זמן הפיתוח, היכולת לרוץ ללא קומפילציה, ורמת אמינות גבוהה הודות לספריות בוגרות ולמנגנוני בדיקה קלים לשימוש. שילוב זה מאפשר לבנות מודל תוכנה מדויק, להשוות אותו מול מימוש RTL, ולוודא התאמה מלאה בין החישובים התיאורטיים לבין ביצועי המעגל החומרתי.

4.4.1. salsa20 script

4.4.1.1. מבוא ומטרת הסקריפט

הסקריפט מממש דוגמה מלאה של אלגוריתם Salsa20 ברמת תוכנה (Python + NumPy) ומשמש לייצור וקטורי בדיקה (test vectors) עבור המימוש החומרתי. הסקריפט מאמת פונקציונליות של האלגוריתם (הצפנה/פענוח), ומפיק קובץ טקסט מסודר עם וקטורי קלט ופלט (key, nonce, counter, constants, cipher, plain, hash matrix, initial matrix) לשימוש בסימולציות, בטסטבנץ חומרתי ובבדיקות אינטגרציה עם Firmware. פורמט קובץ ה- output, salsa_test_vectors, מתואר בנספח א'.

4.4.1.2. מבנה כללי

הסקריפט מחולק לפונקציות קטנות וברורות שכל אחת מהן מממשת חלק מהאלגוריתם או עוזרת בבניית הייצוגים הבינאריים/הקסדצימליים:

- quarter round – מימוש הפונקציה הבסיסית של האלגוריתם salsa20 והרכבת הפונקציות row round, column round ו double round בעזרתו.
- Salsa_hash – מפעיל את double_round מספר פעמים (10) ואז מחבר את התוצאה עם מטריצת הכניסה.
- Four_constant_word – מחזיר את ארבעת המילים הקבועות לפי sel.
- פונקציות עזר ליצירת קלטים: random_key, generate_random_hex_string, random_counter, random_nonce.
- Initial_matrix – מייצר את המטריצה הראשונית ע"פ הקלטים.
- המרות לייצוגים להדפסה: dec2hex_matrix, matrix2Binary_bas, random2Binary_bas, constant2Binary_bas.
- generate_random_test_vectors – הפונקציה המרכזית שמייצרת מספר (num) קלט לפונקציה) וקטורים ושומרת את הפלט לקובץ טקסט.

4.4.3. מצבי ריצה (Modes)

הסקריפט תומך בשתי דרכי ריצה עיקריות, שנקבעות על ידי פרמטר sel של generate_random_test_vectors:

- **מצב אפסים (sel = 0)**
כל רכיבי הקלט (Key, Nonce, Counter) מוגדרים לאפס. מיועד לבדיקה בסיסית: מוודא שה- hash+xor מחרוזתים את הערך המקורי ולוודא את פורמט הפלט. במקרה זה ניתן להגדיר 'num' לערך 1 וכך הסקריפט יפיק טסט יחיד עם כל הקלטים אפסיים.
- **מצב רנדום (sel = 1)**
הסקריפט יפיק 'num' וקטורי בדיקה. בכל וקטור ייווצרו - key, nonce, counter וכן plain רנדומליים (בהתאם להגדרות הפונקציות). מתאים לייצור מספר רב של טסטים לבדיקת פונקציונליות.

ג.4.ב. text2bytes script

הסקריפט נועד לייצר באופן אוטומטי וקטורי בדיקה מלאים עבור האלגוריתם, בדומה לסקריפט salsa20 אך מתוך קובץ טקסט שמתקבל כקלט ולא מתוך הגדרות ידניות. קובץ הפלט מתואר בנספח א'. ישנו שוני בין סקריפטים בכמה פונקציות קריטיות שנועדו לקבל קובץ טקסט ולהמיר אותו למספרים בתצורות מסוימות.

ג.4.ג. salsa_pyserial_uart script

ג.4.ג.1. תיאור כללי – מטרת הסקריפט

הסקריפט מממש ממשק SalsaDeviceUART ליצירת תקשורת סדרתית (UART) מול התקן חומרתי (FPGA). מטרתו העיקרית היא להעניק ממשק נוח ל-Firmware/PC לצורך כתיבה וקריאה של רגיסטרים במכשיר, להציג בפורמט HEX צבעוני את המסגרות הנשלחות ואת התגובות המתקבלות, ולספק כלי בדיקה ידני ומהיר במהלך פיתוח ואימות על הלוח.

ג.4.ג.2. ארכיטקטורה ועקרונות פעולה

המחלקה SalsaDeviceUART, פותחת פורט סידורי (serial.Serial) ומשתמשת בשלוש פעולה עיקריות:

1. enter_active_mode () – שולחת 0xAA לשחרור מכונית המצבים ממצב IDLE.
2. send_and_wait (frame, expected_bytes) – שולחת את ה-frame וממתנה פרק זמן קצר לקריאה אפשרית במידה ונדרש.
3. write_to_device (address, command, data_bytes) – ממשק כתיבת ארבע בתיים.
4. read_from_device (address, command) – ממשק קריאת ארבע בתיים.

הסקריפט מדפיס לצופה המפעיל הודעות צבעוניות (באמצעות ספריית colorama, לצורך ניטור ויזואלי במהלך טסטים).

ג.4.ג.3. הנחות פרוטוקול ו-frame format

הקוד מניח פורמט מסגרת פשוט ועקבי,

- עבור כתיבה-
[address][command][data_byte0][data_byte1][data_byte2][data_byte3]
סך הכל 6 בתיים.
- עבור קריאה-
[address][command] – מחזיר את ארבעת הבתיים מהכתובת.

כמו שכבר אמרנו, לפני כל פעולה כזאת הסקריפט שולח פקודת שחרור מה IDLE, את הבית: 0xAA. כמו כן, ההדפסות מציגות את ה-address וה-command ב-Hex ומציגות את מסגרת השליחה או התשובה.

ג.4.ד. auto_and_menaul_test script

הסקריפט מהווה מנגנון בדיקה אוטומטי לממשק UART מול ה-FPGA, ומאפשר להריץ סדרת טסטים המוגדרים בקובץ JSON, לכתוב ולקרוא רגיסטרים, להשוות תוצאות ולתעד הכל בקובץ לוג מפורט.

סדר הפעולות:

1. טעינת קובץ JSON המכיל את כל קבוצת הרגיסטרים.
2. תקשורת UART מלאה
 - שימוש במחלקה SalsaDeviceUART המיובאת מהסקריפט salsa_pyserial_uart.
 - כתיבה בפורמט Little Endian
 - קריאה ותרגום התוצאה ל-Big Endian לצורך הצגה והשוואה.
3. תמיכה בשתי מצבים
 - מצב Encrypt: מזין קלט וקורא את המידע המוצפן.
 - מצב Decrypt: פלט הקריאה צריך להתאים ל-Plain text- המידע המפוענח.
4. מנגנון השוואה מפורט לכל קריאת רגיסטר
 - המרה ל-BE
 - השוואה מול ערך צפוי
 - הדפסה צבעונית של pass/fail
 - תיעוד מלא בלוג כולל ערכים בשני הפורמטים (BE ו-LE)
5. תיעוד קובץ הלוג עם חותמת זמן.
6. מצב DEBUG אינטראקטיבי - מאפשר להריץ פקודות בזמן אמת

- לקריאה: `r <addr>`
- לכתבה: `w <addr> <value>`
- בדיקות נקודתיות במהלך טסטים
- 7. פענוח התוצאות
 - מרכיב מחדש זרמי נתונים
 - מפענח חלק מהפלט בחזרה ל-UTF-8
 - שומר את התוצאות בקבצי output
- 8. השוואת טקסט ויזואלית על ידי יצירת קובץ HTML
 - הטקסט המקורי
 - הטקסט המפוענח

5.2. Perl Script

Perl היא שפה ותיקה ורבת עוצמה, המיועדת במיוחד למשימות של עיבוד טקסט, ניתוח קבצים, יצירת סקריפטים אוטומטיים ולוגיקה מהירה לבניית כלי בדיקה. אחד מיתרונותיה המרכזיים של Perl הוא התמיכה המובנית בביטויים רגולריים ורמת גמישות גבוהה מאוד בעבודה עם מחרוזות, קבצים ומבני נתונים. יכולות שהופכות אותה לשפה יעילה במיוחד ביצירת קבצי JSON, ניהול קלט/פלט, והכנת וקטורים עבור בדיקות חומרה.

הפרשנות הישירה (ללא קומפילציה) מאפשרת מחזור פיתוח מהיר, והתמיכה הרחבה במודולים חיצוניים מספקת פתרונות מוכנים כמעט לכל צורך. שילוב זה הופך את Perl לכלי נוח לכתבת סקריפטי עזר, בניית תהליכי אוטומציה, ולחיבור יעיל בין מודלים תוכנתיים למימושי RTL.

5.2.1. text_vector_to_json script

הסקריפט, `Text_Vector_to.Json.pl`, נועד לבצע עיבוד מלא ואוטומטי של קובצי וקטורים טקסטואליים הנוצרים מהסקריפט `salsa20.py` ומהסקריפט `text2bytes.py`. הסקריפט מבצע ארבע פעולות מרכזיות:

1. קריאה וניתוח של קובצי וקטורים טקסטואליים.
בקובצי הטקסט ישנם מספר רב של טסטים בראש כל טסט מופיע "Test Number: <>", הסקריפט מזהה כל בלוק כ-Test Case בפני עצמו.
2. חילוץ וסידור של כל רשימות ה-hex.
השליפה נעשית באמצעות Regex חכמים המבטיחים קליטה נקייה של ערכי hex בלבד.
3. בניית רשימות Registers.
הסקריפט יודע עבור כל ווקטור את כתובת הבסיס שלו, ואת כמות ה-dw (מילים).
4. יצירת קובץ json אחד לכל הטסטים.

קובץ json זה ימשך לטובת התקשורת בין המחשב ל-FPGA בצורה אוטומטית ומהירה, ניתן לראות את פורמט הקובץ בנספח ב'.

6.2. Tools

בפרויקט נעשה שימוש במספר כלים תעשייתיים הנפוצים בעולם פיתוח החומרה, לצורך סימולציה, אימות, סינתזה ויישום על רכיבי FPGA. להלן סקירה של הכלים העיקריים.

6.2.1. ModelSim \ QuestaSim - סימולציה ואימות פונקציונלי

ModelSim הוא סימולטור HDL ותיק ונפוץ התומך ב- Verilog, ב- System Verilog ו- VHDL. הכלי מאפשר להריץ סימולציות פונקציונליות ומספק סביבת דיבוג מתקדמת הכוללת צפייה בגלי אותות, נקודות עצירה, הרצת טסטבנצים וביצוע בדיקות עומק לאורך כל מחזור החיים של הפיתוח.

QuestaSim, מבית Siemens/Mentor Graphics, הוא גרסה מתקדמת יותר של ModelSim.

כלים אלו, אינם מוגבלים לפלטפורמות FPGA ספציפיות – משמשים לכל משפחות ההתקנים ביניהם Intel/Altera, Xilinx, Lattice, Microsemi etc. בפרויקט זה השתמשתי בשתי הכלים.

6.2.2. Vivado - פיתוח עבור Xilinx / AMD FPGA

Vivado Design Suite, מבית Xilinx-AMD, הוא הכלי המרכזי לפיתוח, סינתזה, Place & Route, והורדה של חומרה לרכיבי Xilinx מהדורות החדשים (Series-7, UltraScale, Zynq וכו').

השימוש בפרויקט:

- ייבוא קבצי RTL, בדיקת התאמות לסינתזה.
- הפקת דוחות תזמונים (Timing Constraints).
- מיפוי הרכיבים (Place & Route).
- הורדה (programming) לפלטפורמת FPGA ממשפחת Xilinx.
- ניתוח משאבים (LUTs, FFs, RAM, DSP).
- שימוש ב IP PLL

הגדרות כניסות, יציאות, מתחים ו- pin location מוגדרים בקובץ יחיד, קובץ xdc.

6.2.3. Quartus - פיתוח עבור Intel (Altera) FPGA

Intel Quartus Prime הוא סביבת הפיתוח המובילה למשפחות FPGA של Intel/Altera. הכלי מבצע סינתזה, מימוש, בדיקות תזמונים והורדת קבצים למטרות FPGA במשפחות שונות.

שימוש בפרויקט:

- ייבוא קבצי RTL, בדיקת התאמות לסינתזה.
- הפקת דוחות תזמונים (Timing Constraints).
- מיפוי הרכיבים (Place & Route).
- הורדה (programming) לפלטפורמת Intel FPGA.
- ניתוח משאבים (LUTs, FFs, RAM, DSP).

הגדרות כניסות ויציאות מוגדרים בקובץ אחד, קובץ sdc. ואילו הגדרות ה- pin location והמתחים מוגדרים בקובץ נפרד, קובץ qsf. הבדל נוסף מ- Xilinx FPGA, שבו יש קובץ bitstream שצורב את התכנון, ל- intel FPGA, יש שתי אפשרויות:

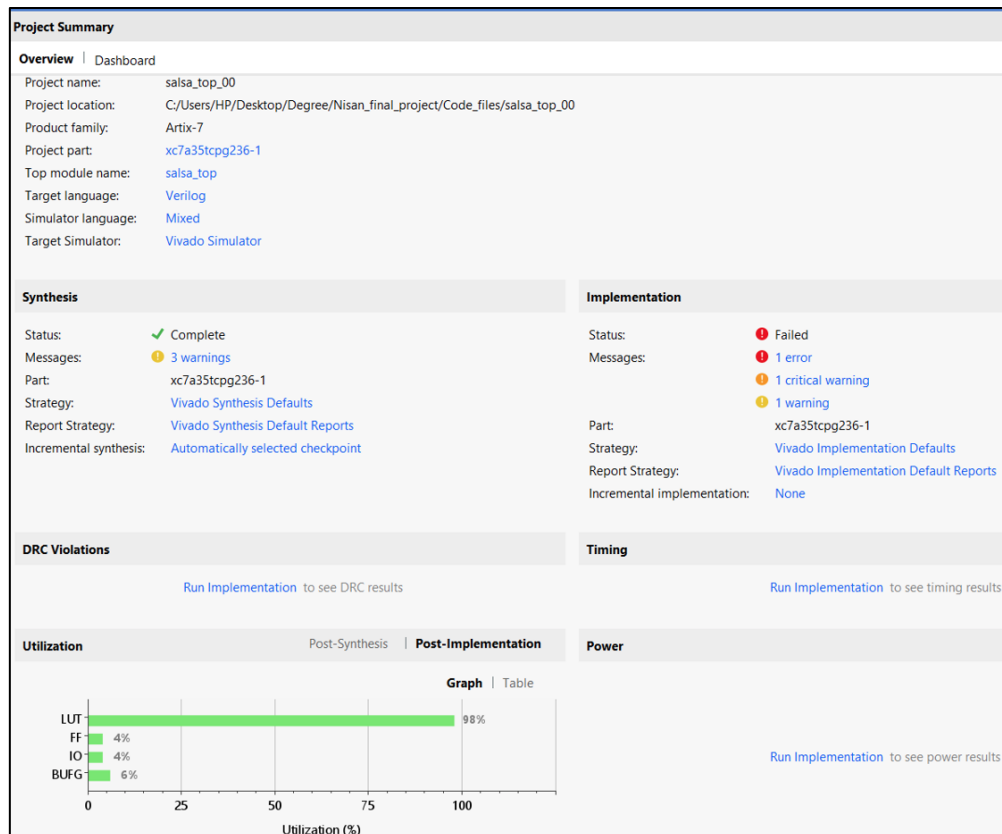
1. קובץ SOF (SRAM Object File), שבעצם המקביל של ה- bitstream, צריבה חד פעמית.
2. קובץ JIC (JTAG Indirect Configuration), שבעצם טוען את התכנון לזיכרון ה- Flash וכך התכנון ישמר על הבורד גם אחרי ניתוק החשמל.

ד. תוצאות

1.1. תוצאות – Basys3

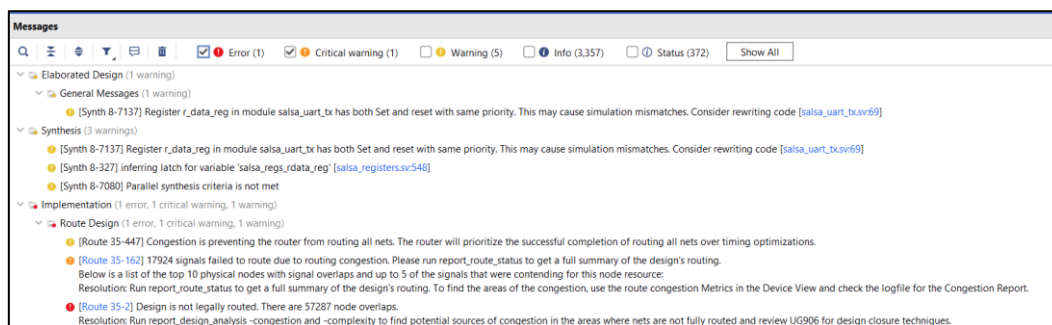
1.1.1. First Implementation

במימוש הראשוני שלי, השתמשתי בשיטת Unrolling, יצרתי כל פונקציה בפני עצמה, והשתמשתי בכמות גדולה מאוד של אינסטנסים ועבודה מקבילית של החומרה תוך רצון לראות היתכנות והפעלה ראשונית. כמו כן, מימוש יחידת ה-hash בצורה קומבינטורית לחלוטין. באופן ראשוני, רק רציתי לראות את ה-P&R שתוכנת Vivado מציאה לי ולכן לא היה לי צורך בהגדרת constraints (קובץ xdc). באיור ד.1, תקציר הפרויקט בתוכנת Vivado, ניתן לראות את אחוזי השימוש ביחידות ה-LUTs של ה-FPGA, אחוזים גבוהים מאוד למימוש זה. כמו כן ניתן לראות שהסטטוס של ה-R&P נכשל.



25- איור ד.1 - תקציר first implementation.

באיור ד.2, נוכל לראות שהפרויקט נכשל עקב מספר עצום של חוטים שלא ניתן היה לעשות להם P&R, מעבר ל-98% מה-LUTs שכבר מיקמנו. כמו כן, התוכנה רצה קרוב ל-16 שעות בניסיון להצליח למקם את כל התכנון עד שהתייאשה.



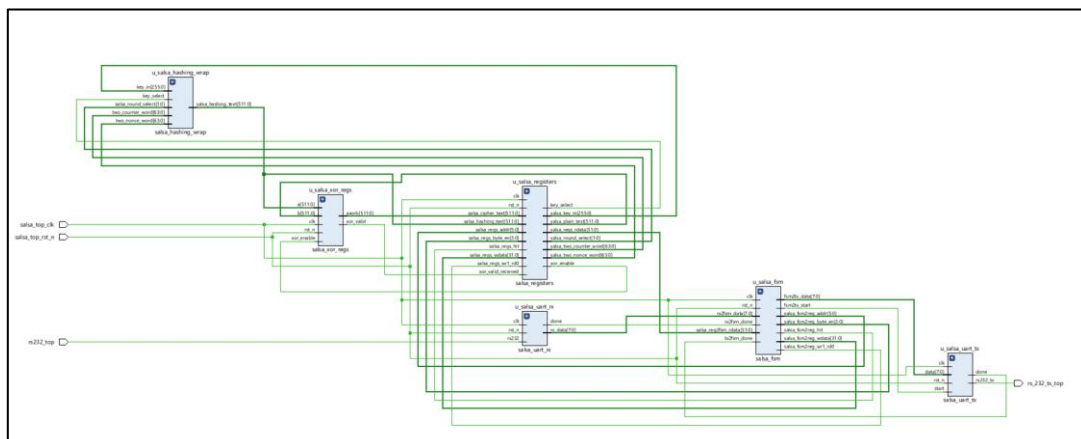
26- איור ד.2 - הודעות מערכת Vivado בניסוי הראשון.

באיור ד.3. מתוארת טבלה מפורטת של ההיררכיה בפרויקט וכמות מספרית של LUTs, Registers ועוד. הנתון המרכזי שניתן לשים לב אליו הוא מספר ה- LUTs שיחידת ה- hashing צורכת- 21,696 ביחס לאפס רגיסטרים, כמצופה מיחידה קומבינטורית מלאה.

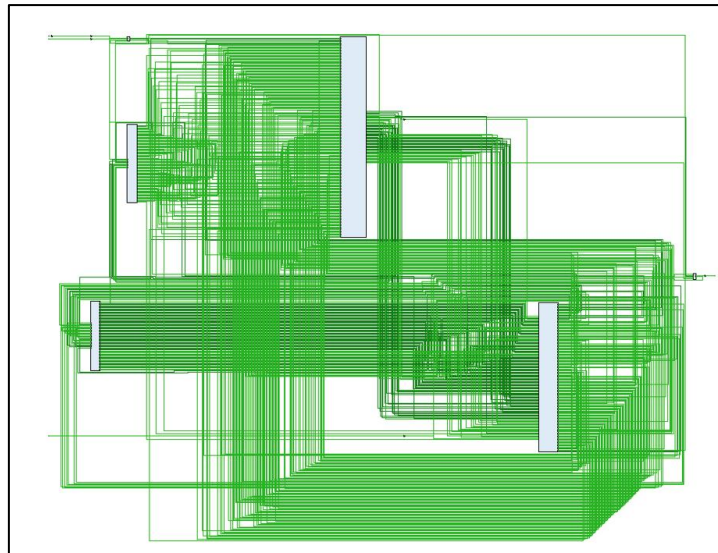
Dataflow Hierarchy Browser				
Name	Blackboxes	IO	Register	LUT
▼ N salsa_top	0	4	1572	23845
u_salsa_fsm (salsa_fsm)	0	0	63	219
u_salsa_registers (salsa_registers)	0	0	934	1684
u_salsa_xor_regs (salsa_xor_regs)	0	0	513	175
▼ u_salsa_hashing_wrap (salsa_hashing_wrap)	0	0	0	21696
▼ u_salsa_hash_function (salsa_hash_function)	0	0	0	21696
> u_double_round_6 (double_round_5)	0	0	0	1310
> u_double_round_5 (double_round_4)	0	0	0	2958
> u_double_round_8 (double_round_7)	0	0	0	1310
> u_double_round_7 (double_round_6)	0	0	0	3049
> u_double_round_9 (double_round_8)	0	0	0	2700
> u_double_round_2 (double_round_1)	0	0	0	1217
> u_double_round_1 (double_round)	0	0	0	3956
> u_double_round_4 (double_round_3)	0	0	0	1530
> u_double_round_3 (double_round_2)	0	0	0	2581
> u_double_round_10 (double_round_0)	0	0	0	1085

איור ד.3 - Dataflow Hierarchy בניסוי הראשון.

באיורים ד.4. ו ד.5. ניתן לראות את ה- RTL Schematic לפני סינטזה ואחרי סינטזה.

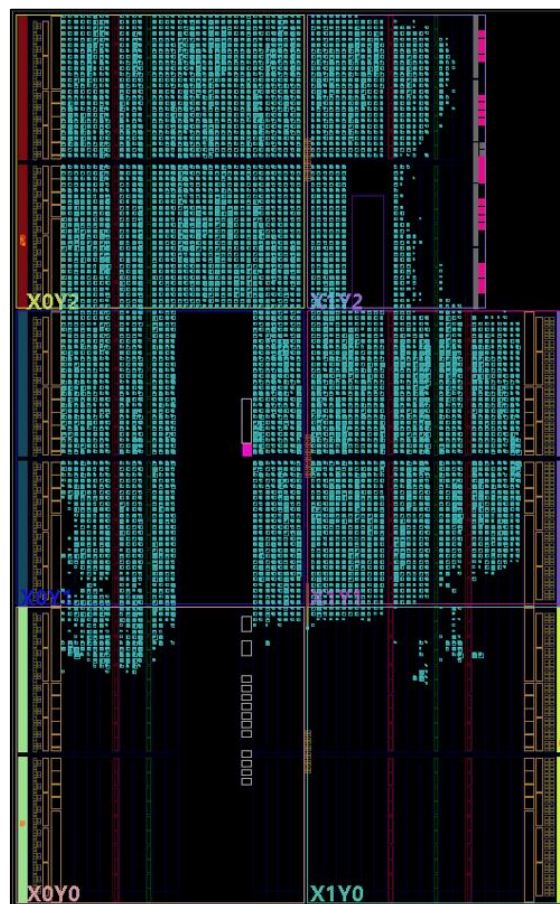


איור ד.4 - RTL Schematic לפני סינטזה בניסוי הראשון.



איור ד. 5 - RTL Schematic אחרי סינטזה בניסוי הראשון.

באיור ד. 6. מתואר ה-P&R על רכיבי ה-FPGA.

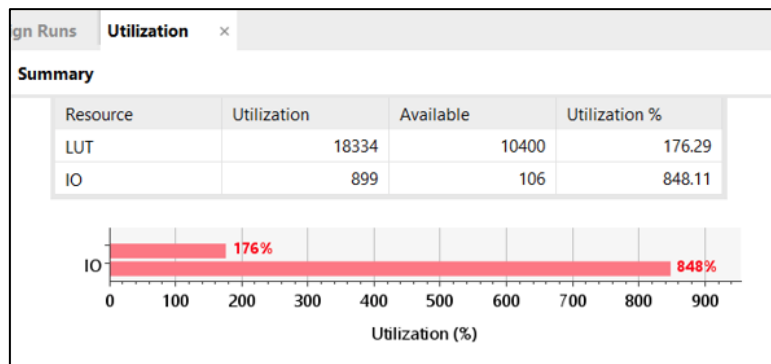


איור ד. 6 - Implementation Device בניסוי הראשון.

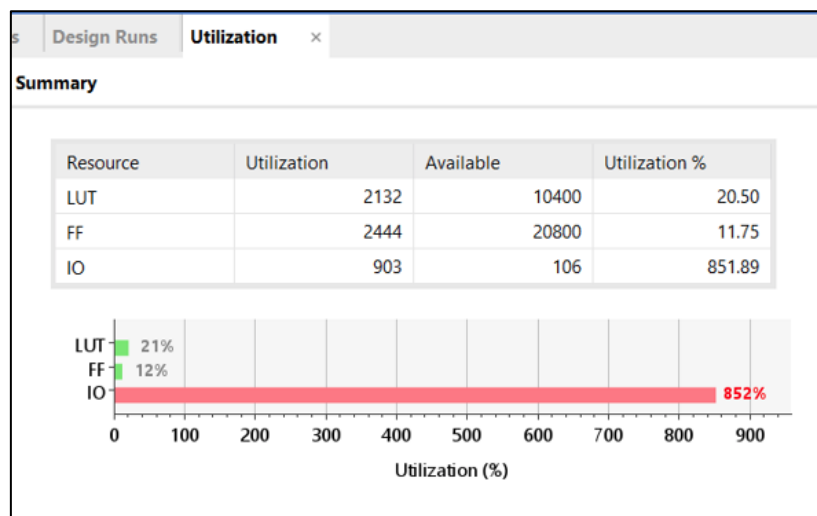
ד.1.1. Interactive Hash VS Unrolling Hash

לאחר שהתברר כי קיימת מגבלת משאבים במימוש ובכריב הנוכחי, הוחלט לשנות את יחידת ה- Hashing למימוש בסגנון Interactive. בשלב זה הוחלט לבצע שינוי חלקי בלבד ביחידה, ולא לשנותה במלואה. בהמשך תפורט ההחלטה לעבור למימוש מלא של היחידה, כפי שמתואר בפרק ג.2.ב.

בשיטה זו הושג חיסכון במשאבי חומרה באמצעות שימוש ברגיסטרים (Flip-Flops) ובמכונות מצבים. בעת ביצוע תהליך ה- Place & Route (P&R), לצורך השוואה בין שתי היחידות, לא נלקחו בחשבון כניסות ויציאות היחידות, שכן במערכת המלאה הן אינן מהוות ממשקי קלט/פלט חיצוניים, אלא חלק פנימי מהמערכת. לפיכך, אין להתייחס למספר ה- I/Os המוצגים באיורים ד.7 ו- ד.8.



איור ד.7 - 31 Utilization report Unrolling hash

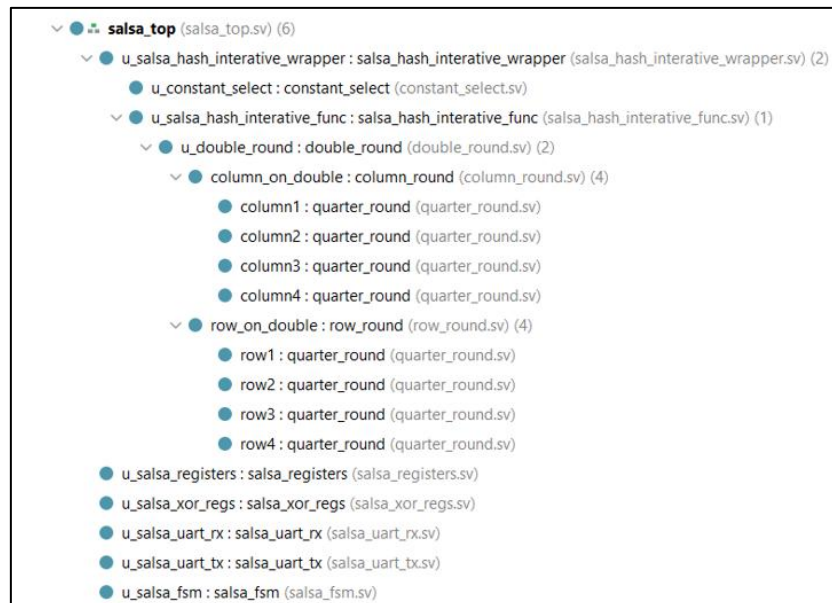


איור ד.8 - 32 Utilization report Interactive hash

השימוש ב- LUTs הוקטן פי 8.6, והשימוש ב- Flip-flops מהווה רק 12% מהכמות המקסימלית.

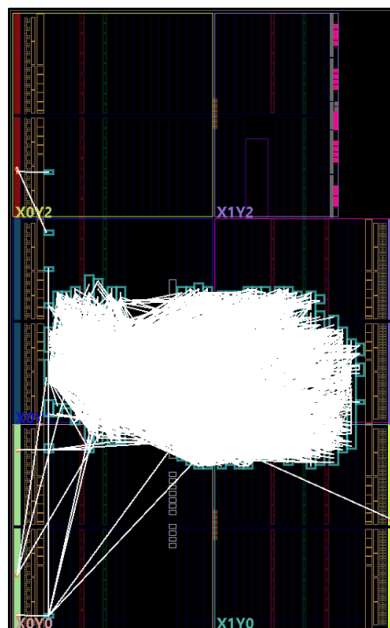
ד.1.2. Second Implementation

בניסיון זה נעשה שימוש במימוש חלקי בסגנון interactive של יחידת ה-hash, כמתואר באיור ד.9. בו ניתן לראות את היררכיית המודולים, שבה עדיין קיימים מספר מופעים חוזרים של פונקציות הבסיס.



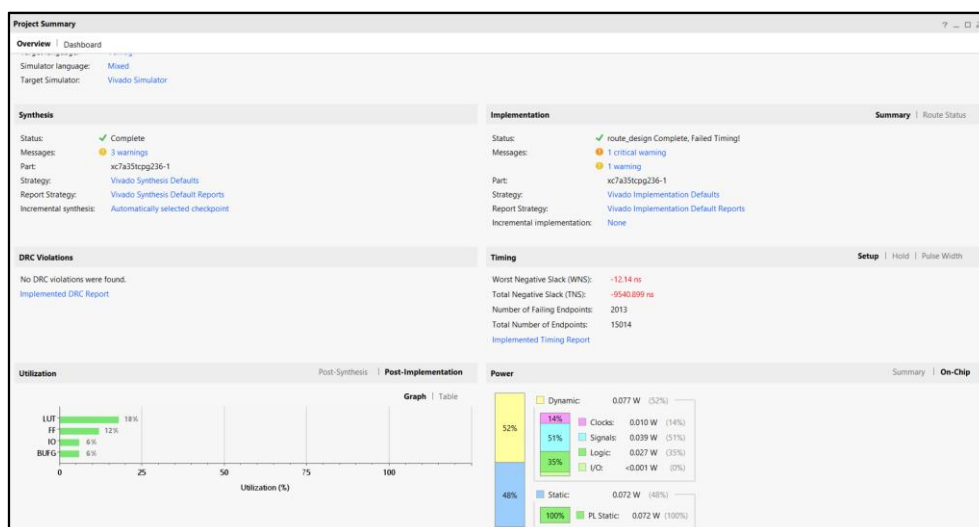
איור ד.9 - היררכיית הפרויקט בניסיון השני.

באיור ד.10. ניתן לראות את צמצום ההתפרסות על ה-FPGA ב-P&R.

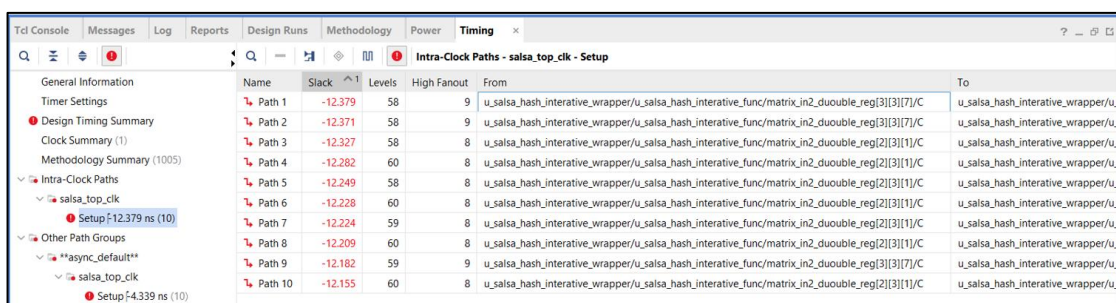


איור ד.10 - Implementation Device בניסיון השני.

באיור ד.11. מוצג תקציר הניסוי השני. אומנם מבחית מקום פתרנו את הבעיה אך נתקלנו בבעיות זמנים אותם ניתן לראות באיור ד.12.



35- איור ד. 11 - תקציר ניסיון שני.

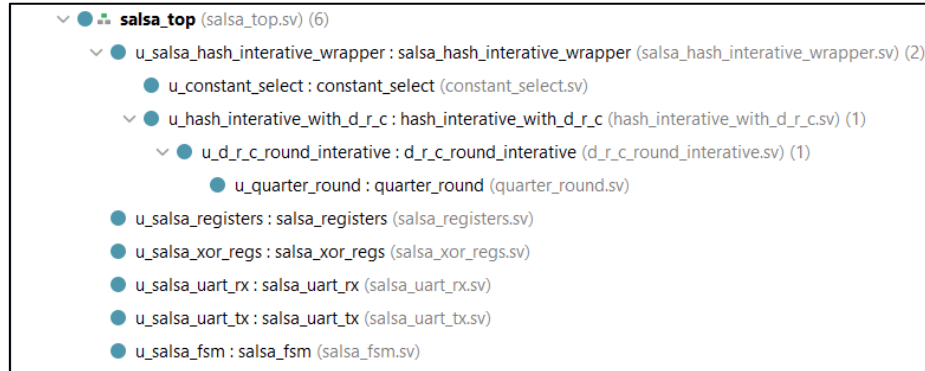


36- איור ד.12 - *Timing Report* ניסיון שני.

עקב בעיות הזמנים הוחלט לממש את כל יחידת ה-hash בצורת כתיבה מסוג Iterative.

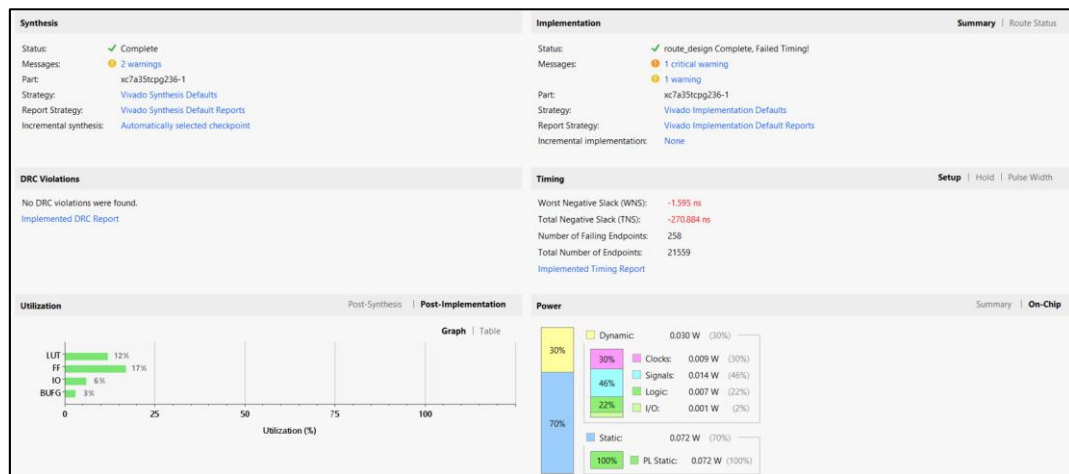
ד.1.3. Third Implementation

בניסיון השלישי, יחידת ה-hash מומשה כמוזכר בפרק ג.2.ב. באיור ד.13. ניתן לראות את היררכיית המודולים בניסיון זה, נקי מאינסטנסים מרובים של פונקציות הבסיס.



איור ד.13 - היררכיית הפרויקט בניסיון השלישי.

באיור ד.14. מוצג תקציר הניסוי השלישי. גם כאן כמצופה חסכנו עוד מקום אך בעיית הזמנים לא נפתרה.



איור ד.14 - תקציר ניסיון שלישי.

ה-Worst Negative Slack (WNS) שנמדד במערכת הוא "1.595 ns-". כלומר, עבור אילוף שעון של 10ns הנתיב הקריטי אינו עומד בדרישות התזמון ודורש תוספת של עד "1.595 ns". בהתאם לכך, במשוואות (15-17) מחושב תדר העבודה המקסימלי האפשרי של המערכת.

$$Basy3CLK = 100MHz \Rightarrow 1 \text{ Cycle} = \frac{1}{100 \cdot 10^6} = 10ns \quad (15)$$

$$10ns + 1.595ns = 11.595ns \quad (16)$$

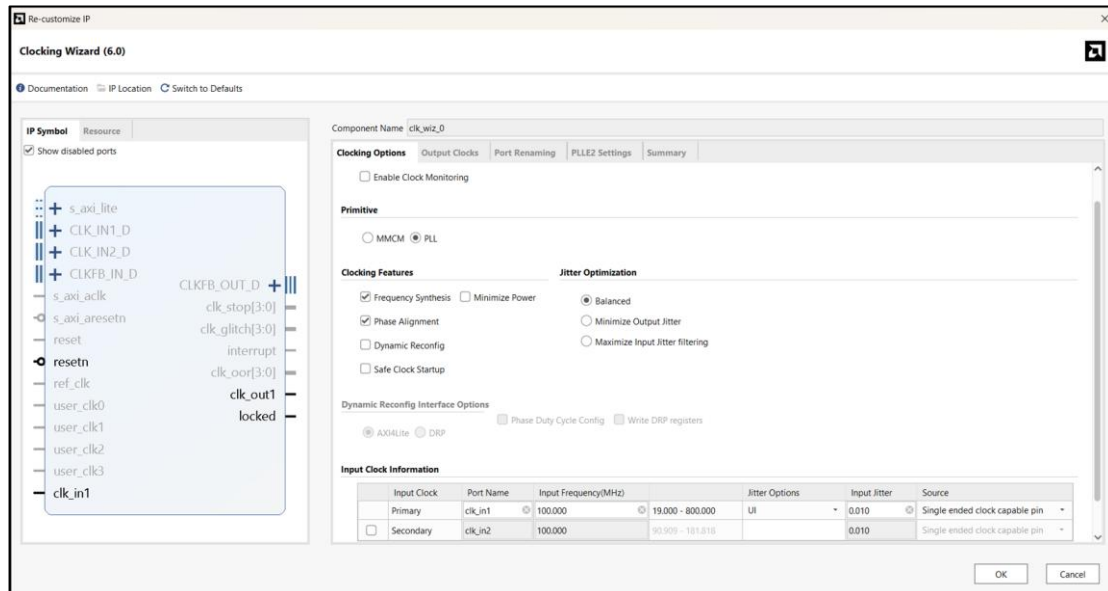
$$newCLK_{MAX} = \frac{1}{11.595 \cdot 10^{-9}} = 86.24MHz \quad (17)$$

בהתבסס על ה-Worst Negative Slack שנמדד, תדר העבודה המקסימלי של המערכת (Fmax) מוערך בכ- 86MHz, על מנת לעמוד באילוצי התזמון ללא הפרות. בהתאם לכך, יש צורך בשימוש ב-PLL (Phase Locked Loop) לשם התאמת והפקת תדר שעון מתאים עבור המערכת.

ד.1.ה. Vivado PLL IP

ל- FPGA יש מקור שעון יחיד בתדירות 100MHz ולכן כדי לייצר ממנו 80MHz נשתמש ב- PLL IP של Vivado.

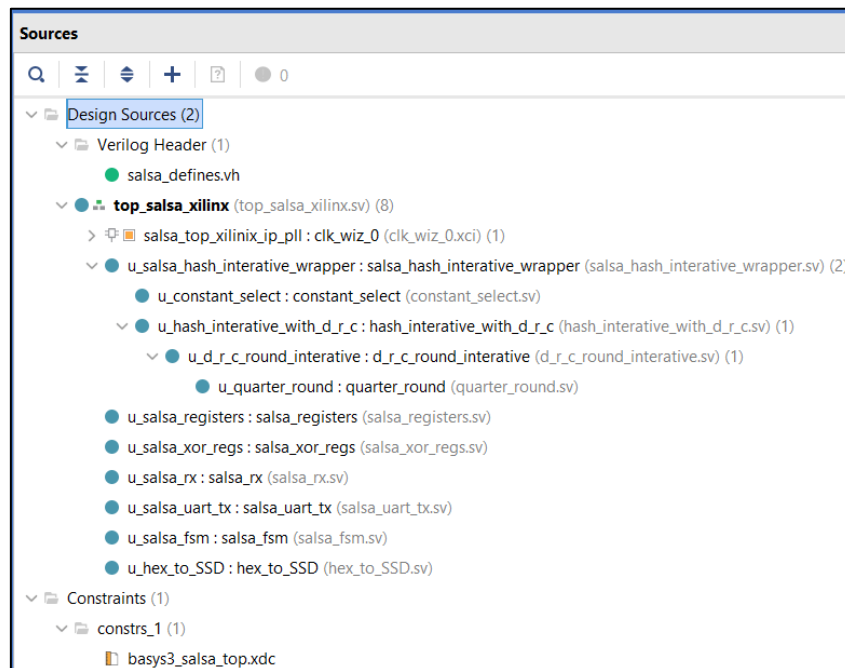
PLL (Phase Locked Loop), הוא רכיב חומרה ייעודי שמאפשר ייצוב, הכפלה, חלוקה וסנכרון תדרים. IP (Intellectual Property), רכיב חומרה מוכן מראש, מודל לוגי שניתן לשלב בקלות בתוך הפרויקט כדי לחסוך בזמן, להימנע מטעויות וליהנות מיישום שכבר נוסה ונבדק במקום לתכנן דבר מורכב מאפס. במקרה הזה, בחרתי להשתמש ב- PLL של Vivado, כמתואר באיור ד.15.



39- איור ד.15 - הגדרות PLL IP של Vivado.

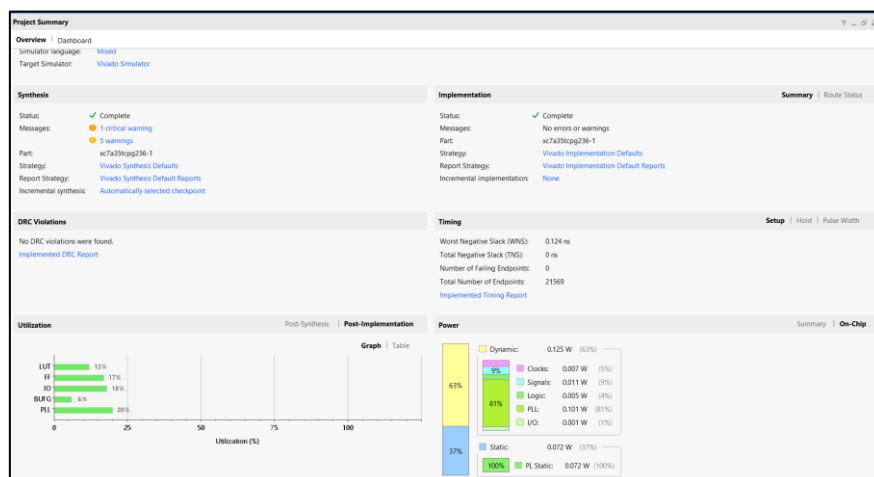
ד.1.1. The Last Implementation

במסגרת מימוש זה שולב קובץ ה-XDC, בהתאם למתואר בפרק ג.2.ז. איור ד.16 מציג את היררכיית המודולים המלאה של המערכת, הכוללת הן את קובץ ההגדרות (defines) והן את קובץ ה-xdc.



איור ד.16- היררכיית הפרויקט בניסיון האחרון.

באיור ד.17. מוצג תקציר הפרויקט הסופי, ניתן לראות שאין בעיית זמנים ושהתווסף PLL למערכת שתופס 20% מסך ה- PLLS שניתן להשתמש בהם על ה- FPGA.



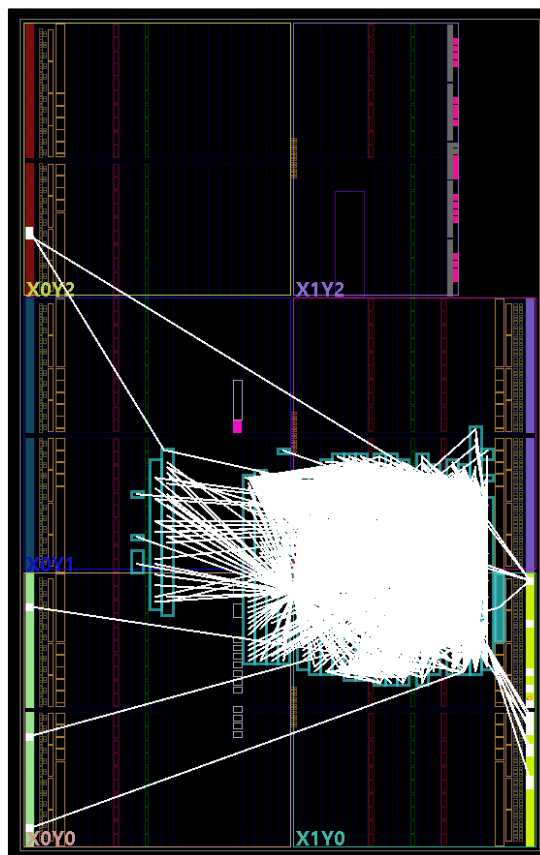
איור ד.17 - תקציר הפרויקט הסופי.

באיור ד.18. מוצג תקציר דוח הזמנים של המערכת, ללא בעיות זמנים ואפילו אם נתוני זמנים טובים.

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 0.124 ns	Worst Hold Slack (WHS): 0.008 ns	Worst Pulse Width Slack (WPWS): 2.633 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 21569	Total Number of Endpoints: 21569	Total Number of Endpoints: 7224	
All user specified timing constraints are met.			

איור ד.18 - תקציר דוח הזמנים הסופי.

באיור ד.19. מוצג פריסת התכנון על ה-FPGA, ניתן לראות בצורה ויזואלית את כמות השטח שהתכנון צריך.



איור ד.19 - Implementation device הפרויקט הסופי.

ד.1.1. אינטגרציית המערכת

באיור ד.20. מצורף קטע טקסט קצר מתוך הפרק הראשון של הארי פוטר ואבן החכמים, קטע טקסט זה מהווה קובץ input לסקריפט "text2bytes.py", שמניב קובץ טקסט, "vectors_from_harry2.txt", המכיל 12 בלוקים של נתונים להצפנה.

כמו כן, חשוב לציין שלאורך כל הפעילות ישנו קובץ לוג המפרט כל גישת כתיבה או קריאה מהמחשב לבורד ולהיפך, קובץ מפורט מאוד עם בדיקות וציון עובר או נכשל לכל טרנזקציה, קובץ לוג זה נמצא ב-git עם כל שאר הקבצים של הפרויקט.

```
1 Chapter 1 - The Boy Who Lived.
2 Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say that they were perfectly
3 normal, thank you very much. Mr. Dursley made drills. He was a big, beefy man with hardly any
4 neck, although he did have a very large moustache. Mrs. Dursley was thin and blonde and had twice
5 the usual amount of neck, which came in very useful as she spent so much of her time spying on the
6 neighbours. The Dursleys had a small son called
7 Dudley and in their opinion there was no finer boy anywhere. Mrs Dursley had a sister called Lily Potter. She
8 and her husband James Potter had a son called Harry Potter. They lived far from the Dursleys and
9 did not speak to them much. They did not get along.
```

44- איור ד.20 - קטע טקסט מתוך הארי פוטר ואבן החכמים.

קובץ הטקסט הזה, משמש כקובץ input לסקריפט "text_vector_to_json.pl", שמניב קובץ Json עם 12 בלוקים מסודרים בפורמט המוגדר בנספח ב', כמתואר בפרק ג.5.

ע"י שימוש בפונקציה "auto_and_manual_test_3.py", שקורא את קובץ ה-Json ומתקשר עם ה-FPGA, נקבל את קטע הטקסט המוצפן המצורף באיור ד.21.

```
1 JDDCvAPI+EYufSNQQFJR6rFkaa7wa2ga3LQBfbAZAK1UPC8zuzeXstDVoEOlwZl8
2 cJP8oK4lMnIoGcOWntozAfHsWqyyJP3ce+k/8zAgC0dN7m0yibv9T8ZYI4AoFVYo
3 8ZPI6uUENTL39bIsJ0XQwwg5QlJmc3pxlikPxF6JCTkEVFTgeuCi78AZclsUwY3g
4 SWnrCPr4uuC9ikak4LxACQPEStcmdQBx2v9NR8uYSFu+OmnNRjCB3/ZYdUxI6i7
5 hzgiHqDfCuyHJOrlwhjxIRpjw32oRdSDZvCIkUuo7YEuotkPmZXatobDg+yA4IB
6 HbcA50JSo2j4jgfft3VBckux6Acr0Q73h47Nlnqh5Aq9CWLGSpmASyzd9HawLzQ
7 hkdHwAB2ZXRTDAj//Lh0M/5EhQpfVHZmi2I2woQinqt9qm3863wbIcgo7FAQ4Xvq
8 RbDC2ayW9hexD/rCfeVx5qkMsT2XZyrqn4jQy1WV7vFu8sBCOEKfQBkjjw0X7RY8M
9 jaFmq10ASLwrJCzYRKs2jGcmYdKYoeYvwpqWYc0XFz+eWykn4mv3BRNW5tp4si7c
10 lK5VSSm4QFotEYdreimQIqxZC4sDqh7EhQXEXHfLkU19leeyFAyviGUdgOe5DXyU
11 czaWzJLsZXLiJjHwaQoGf7NKO/2kQtSdbJ4WwblpUPUWn5luOfIXtAj8Si0iJIRh
12 DxWUJrL7QhhB6GJ7vqUOfvQFL01Vpy4z089cd1ShiyeoZ/y4hYtQIIT4ng7A5IkV
13 FO8fKmhkpeXZqIy/SyEftSbBLAcavULpoy7IHDvQm9mlzCCaRt4YL19UqddAhCXW
14 I9mG280spmjx+YZ0urpcpZ6sYd0pQF/WFGSpnWwf84Gt1bwT8RlghqOwLKvfcRNx
15 GnKS0Pr7wrGaoxZMriw7pwYqT55QOWGA2ESghZyKS+U3Aq6qxgRBaPwB7rD41ng1
16 lymopKpgwok00vMz1lB10AEhtRtZ4ospsH+WAKHZCaz9Wv3fhNiE5kgRmbu/1lhU
```

45- איור ד.21 - קטע הטקסט Harry potter המוצפן.

לאחר גמר ההצפנה, המשתמש מתבקש לבחור האם לפענח את הטקסט המוצפן ולקבל עותק של הטקסט המפוענח כפי שמוצג באיור ד.22.

```
1 Chapter 1 - The Boy Who Lived.
2 Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say that they were perfectly
3 normal, thank you very much. Mr. Dursley made drills. He was a big, beefy man with hardly any
4 neck, although he did have a very large moustache. Mrs. Dursley was thin and blonde and had twice
5 the usual amount of neck, which came in very useful as she spent so much of her time spying on the
6 neighbours. The Dursleys had a small son called
7 Dudley and in their opinion there was no finer boy anywhere. Mrs Dursley had a sister called Lily Potter. She
8 and her husband James Potter had a son called Harry Potter. They lived far from the Dursleys and
9 did not speak to them much. They did not get along.
```

46- איור ד.22 - קטע הטקסט Harry potter המפוענח.

כמו כן, לאחר הפענוח, המשתמש נשאל האם רוצה לקבל קובץ html לצורך השוואת הקובץ המקורי אם הקובץ המפוענח בצורה ויזואלית וברורה כפי שמוצג באיור ד.23.

Decoded	
1	Chapter 1 - The Boy Who Lived.
2	Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say that they were perfectly
3	normal, thank you very much. Mr. Dursley made drills. He was a big, beefy man with hardly any
4	neck, although he did have a very large moustache. Mrs. Dursley was thin and blonde and had twice
5	the usual amount of neck, which came in very useful as she spent so much of her time spying on the
6	neighbours. The Dursleys had a small son called
7	Dudley and in their opinion there was no finer boy anywhere. Mrs Dursley had a sister called Lily Potter. She
8	and her husband James Potter had a son called Harry Potter. They lived far from the Dursleys and
9	did not speak to them much. They did not get along.
10	

Legends

Colors	Links
Added	(f)irst change
Changed	(n)ext change
Deleted	(t)op

Original	
1	Chapter 1 - The Boy Who Lived.
2	Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say that they were perfectly
3	normal, thank you very much. Mr. Dursley made drills. He was a big, beefy man with hardly any
4	neck, although he did have a very large moustache. Mrs. Dursley was thin and blonde and had twice
5	the usual amount of neck, which came in very useful as she spent so much of her time spying on the
6	neighbours. The Dursleys had a small son called
7	Dudley and in their opinion there was no finer boy anywhere. Mrs Dursley had a sister called Lily Potter. She
8	and her husband James Potter had a son called Harry Potter. They lived far from the Dursleys and
9	did not speak to them much. They did not get along.
10	

47- איור 23. ד - השוואה בין הטקסט Harry potter המפוענח לטקסט המקורי (html).

ד.2. תוצאות – DE10-Lite

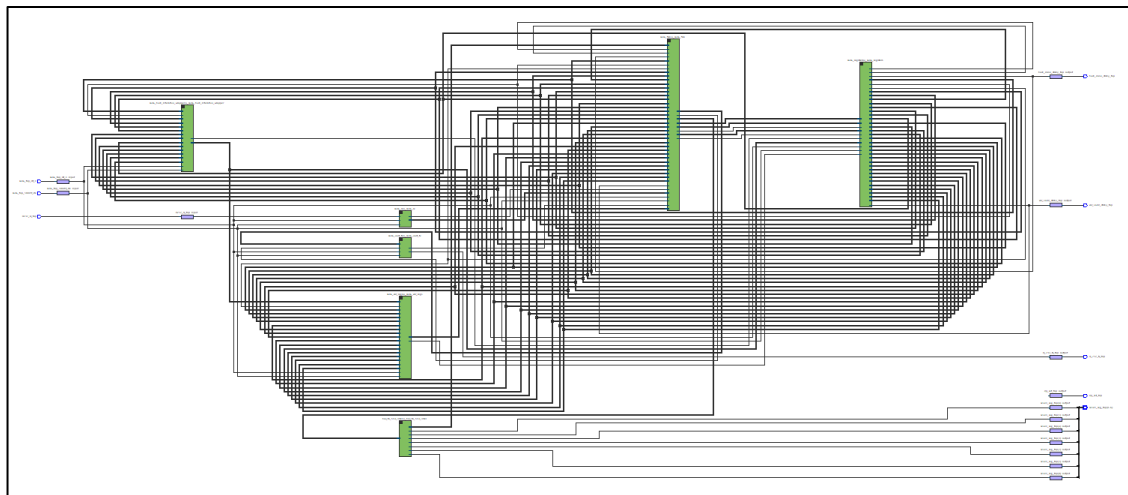
ד.2.א. Implementation

באיור ד.24. ניתן לראות את תקציר הפרויקט, המכיל שמות, הגדרות כלליות והגדרות זמנים.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sun Nov 23 23:39:52 2025
Quartus Prime Version	23.1std.1 Build 993 05/14/2024 SC Lite Edition
Revision Name	top_salsa_intel_00
Top-level Entity Name	top_salsa_intel
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	8,125 / 49,760 (16 %)
Total registers	7202
Total pins	14 / 360 (4 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 288 (0 %)
Total PLLs	0 / 4 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

איור ד.24 - תקציר הפרויקט DE10-Lite.

באיור ד.25. ניתן לראות את ה- RTL schematic של המערכת.



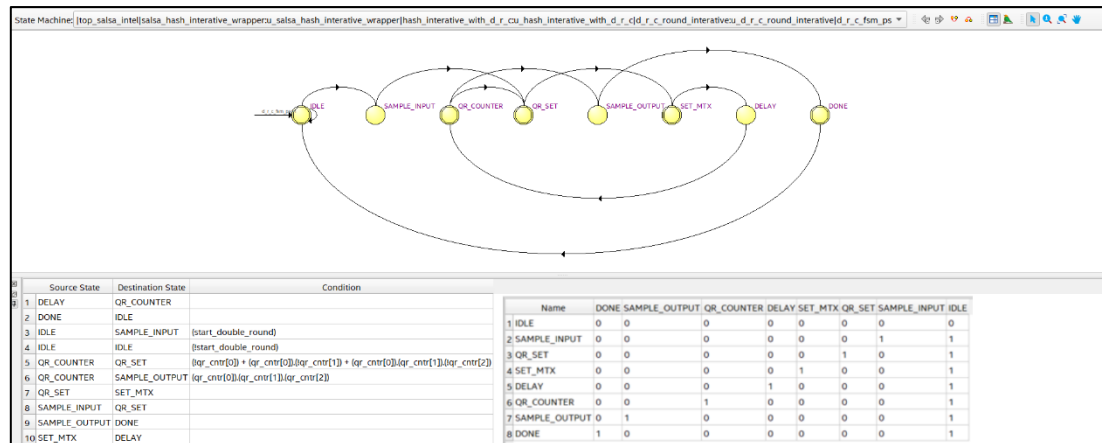
איור ד.25 - RTL schematic DE10-Lite.

באיור ד.26. מוצגת טבלת דוח הזמנים של המערכת מתוך Quartus, תקציר הזמנים מראה נתונים טובים מאוד על המערכת והיכולות שלה.

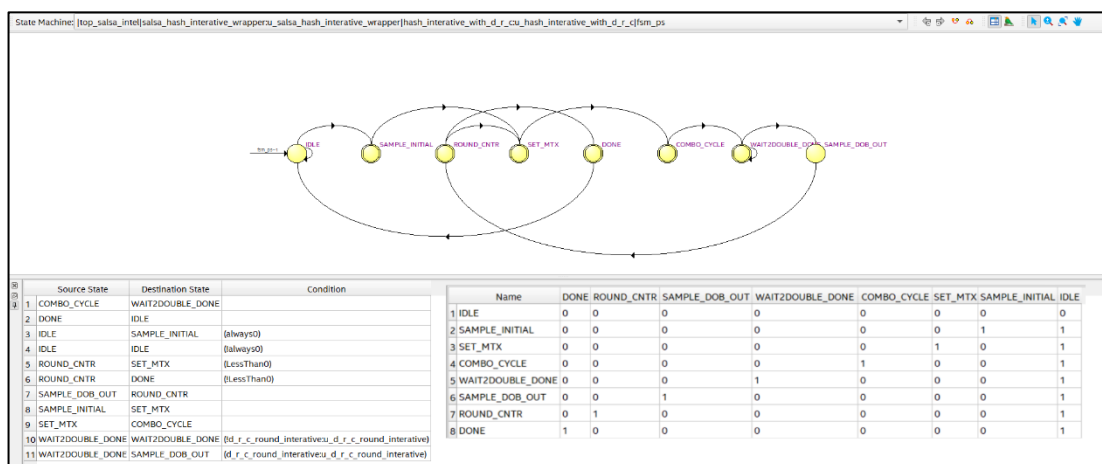
Multicorner Timing Analysis Summary						
<<Filter>>						
	Clock	Setup	Hold	Recovery	Removal	Minimum Pulse Width
1	Worst-case Slack	1.426	0.148	14.817	0.552	9.297
1	salsa_top_50mhz_clk	1.426	0.148	14.817	0.552	9.297
2	Design-wide TNS	0.0	0.0	0.0	0.0	0.0
1	salsa_top_50mhz_clk	0.000	0.000	0.000	0.000	0.000

איור ד.26 - דוח זמנים DE10-Lite.

כמו כן, ניתן לראות באיורים ד.27. ו ד.28. את דיאגרמת הבועות של ה- FSM הפנימיים של המערכת, על פי ניתוח תוכנת Quartus.

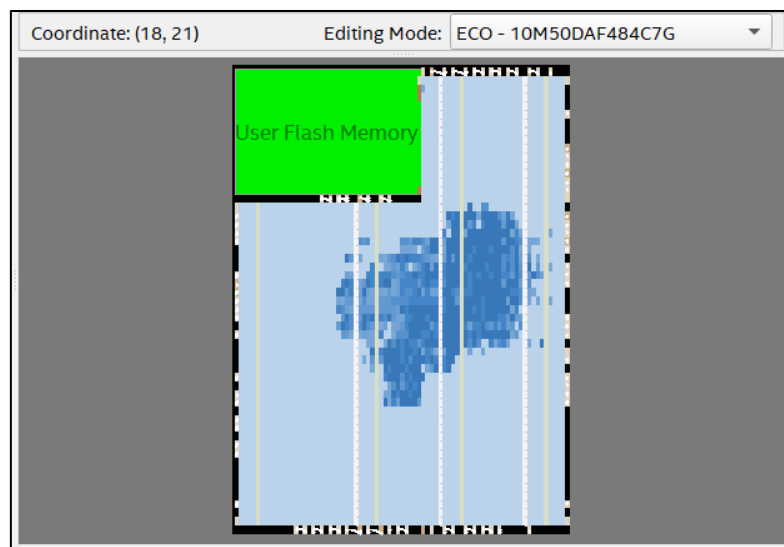


איור 27. d_r_c_interative FSM by Quartus



איור 28. hash_interative FSM by Quartus

באיור 29. מוצגת פריסת התכנון על ה-FPGA, ניתן לראות שהפרויקט שלנו הוא חלק מאוד זעיר מהיכולות של ה-FPGA.



איור 29. chip planner DE10-Lite

ד.2.ב. אינטגרציה

באיור ד.30. מצורף קטע טקסט קצר מתוך הקדמה לפניני הלכה (שבת כרך א'), קטע טקסט זה מהווה קובץ input לסקריפט "text2bytes.py", שמניב קובץ טקסט, "vectors_from_pninahalacha.txt", המכיל 13 בלוקים של נתונים להצפנה.

כמו כן, חשוב לציין שלאורך כל הפעילות ישנו קובץ לוג המפרט כל גישת כתיבה או קריאה מהמחשב לבורד ולהיפך, קובץ מפורט מאוד עם בדיקות וציון עובר או נכשל לכל טרנזקציה, קובץ לוג זה נמצא ב-git עם כל שאר הקבצים של הפרויקט.

In the Torah, Shabbat is one of the commandments with the most principles, generalities and details. That's why Rav Eliezer Melamed has decided to dedicate two volumes to it in the Pnina Halacha series of books.

In this first volume, you will discover the deep spiritual foundations of Shabbat, but also the laws of preparation, prayer, kidush, the reasons why there are 39 forbidden works, and the detailed study of the first forbidden works.

In a clear, concise and methodical style, this book will enlighten the reader who wishes to discover or deepen his knowledge of Shabbat.

In addition, the book refers to the different customs among the people of Israel, and also focuses on all the contemporary questions about Shabbat in connection with new technologies.

54- איור ד.30 - קטע טקסט מתוך פניני הלכה.

קובץ הטקסט הזה, משמש כקובץ input לסקריפט "text_vector_to_json.pl", שמניב קובץ Json עם 13 בלוקים מסודרים בפורמט המוגדר בנספח ב', כמתואר בפרק ג.5.א.

ע"י שימוש בפונקציה "auto_and_manual_test_3.py", שקורא את קובץ ה-Json ומתקשר עם ה-FPGA, נקבל את קטע הטקסט המוצפן המצורף באיור ד.31.

```
1 mLHM53CJ8NyY7o3YTI4bm5XLI22+Bcl9Q5i55MKY9ltyKjYlqUzWsnnrNggoQj7H
2 OFo0iXwOmSzmecZuPL7jZzymEvKNzzTDloG16aieW/WUvThArRrtbTRw91bUP8s
3 LVn9IrrciI8v0oyTuMcFjAURrMiZ17u4GjIgnkUNTHz+CiLtQLkmHwBc/CP0OGbe
4 ps8geGLOynlWcespLwmbh65JfSp5ocecRm9o8AFVvtzsqmznzsiLbRoarFfUuG64
5 vOu6rCpkyBR9rrLILI/RkzrEgA/FePVA6aozui526Ky9eeAZSme6C+n5Oq5pYkEO
6 yahlBzheQ2bGKahk8AwIwTbCZ9Y9ogFfU1OZzdDbglueXfgnY4OgNbNfaBtAUxy4
7 ncHlQAVRw6rQJyyWIKJtvYGIvBvL2Tq9KAlIE33dYpPlaR0g1rmqzNuc5uFAl2uv
8 HRL4gytVGmRBah7B91PPFXnP/krndsEha4xohpWnVQlgsqgWRAoxku8cLwh91f23
9 IldVi84a6/X4R1GNVCqSvue/YoBj7Pg8xQpLjGQ7ep2ibDyAknXHdP18loH1a7mO
10 g6mYsvtmOI9eFwrJloBOCw//xS4VTO2DCmW2a2D4SguOpj2K3JtaULwivKfpV6DD
11 58WusYxoePhXYhQxE3oVUQzi9OczTU3z/0/0zYHnICd37KEEKsQb9dMaezJJpRos
12 VUhw3qEavXBbqqDOIweGGlu9DFmvXfy/WnkftDSzEJoM286PnynXz6T/IQINNaBa
13 WBNTZBeEfjhwrhyF2CWHZUWfugZhlwzaVw7tvs+p4dolrOJALU+ke7nNnkX8Bk+0
14 xkucq47s1H1o5mxziMe/LYYNhk/oOjr3bHO47uRJ22Wlia3zAgPes0VALVdrI2JK
15 a3l+RXFGjwGBfNqB5akz8dnv10PEP7mj0FK+1ryc4isMdnK4GpEpFepYHPMqPOzR
16 rugqe+dZbUWf1VMkaSMFDn/nQEiJEZpoD6CLF2kKbVfhEa6AjKObK57R4W7KrqAN
17 w4lbp9jh6xeyTTTyov77H56A6KHuDyT9oec+BCO7RNYeA/jLpDRLFLnmeNEchU7t
18 lBsY7A51s1LrHJT29zsz/A==
19
```

55- איור ד.31 - קטע פניני הלכה המוצפן.

לאחר גמר ההצפנה, המשתמש מתבקש לבחור האם לפענח את הטקסט המוצפן ולקבל עותק של הטקסט המפוענח כפי שמוצג באיור ד.32.

```
1 In the Torah, Shabbat is one of the commandments with the most principles,
2 generalities and details. That's why Rav Eliezer Melamed has decided to dedicate two volumes to
3 it in the Pnina Halacha series of books.
4 In this first volume, you will discover the deep spiritual foundations of Shabbat,
5 but also the laws of preparation, prayer, kidush, the reasons why there are 39 forbidden works,
6 and the detailed study of the first forbidden works.
7
8 In a clear, concise and methodical style, this book will enlighten the reader who wishes to
9 discover or deepen his knowledge of Shabbat.
10 In addition, the book refers to the different customs among the people of Israel,
11 and also focuses on all the contemporary questions about Shabbat in connection with new technologies.
```

56- איור ד.32 - קטע פניני הלכה המפוענח.

כמו כן, לאחר הפענוח, המשתמש נשאל האם רוצה לקבל קובץ html לצורך השוואת הקובץ המקורי אם הקובץ המפוענח בצורה ויזואלית וברורה כפי שמוצג באיור ד.33.

Decoded	
1	In the Torah, Shabbat is one of the commandments with the most principles,
2	generalities and details. That's why Rav Eliezer Melamed has decided to dedicate two volumes to
3	it in the Pninah Halacha series of books.
4	In this first volume, you will discover the deep spiritual foundations of Shabbat,
5	but also the laws of preparation, prayer, kidush, the reasons why there are 39 forbidden works,
6	and the detailed study of the first forbidden works.
7	
8	In a clear, concise and methodical style, this book will enlighten the reader who wishes to
9	discover or deepen his knowledge of Shabbat.
10	In addition, the book refers to the different customs among the people of Israel,
11	and also focuses on all the contemporary questions about Shabbat in connection with new technologies.

Legends	
Colors	Links
Added	(f) first change
Changed	(n) ext change
Deleted	(t) op

Original	
1	In the Torah, Shabbat is one of the commandments with the most principles,
2	generalities and details. That's why Rav Eliezer Melamed has decided to dedicate two volumes to
3	it in the Pninah Halacha series of books.
4	In this first volume, you will discover the deep spiritual foundations of Shabbat,
5	but also the laws of preparation, prayer, kidush, the reasons why there are 39 forbidden works,
6	and the detailed study of the first forbidden works.
7	
8	In a clear, concise and methodical style, this book will enlighten the reader who wishes to
9	discover or deepen his knowledge of Shabbat.
10	In addition, the book refers to the different customs among the people of Israel,
11	and also focuses on all the contemporary questions about Shabbat in connection with new technologies.

57- איור ד.33 - השוואה בין הקטע פניני הלכה המפוענח לקטע המקורי (html).

ה. דיונים

ה.1. סוגיית ה-Endianness

במהלך השוואת תוצאות הניסויים למול סימולציות ה-RTL ולמול הפלטים הרשמים של Salsa20, עלתה חשיבותו הקריטית של נושא סדר הבתים (Endianness) - אחת הנקודות הרגישות והמהותיות ביותר במימוש חומרתי של האלגוריתם. Salsa20, בדומה לאלגוריתם ChaCha, מוגדר במקור כאלגוריתם הפועל במבנה Little-Endian, הן ברמת טעינת המילים למטריצה והן ברמת פעולות ה-QuarterRound. כל שינוי באופן פירוש הבתים - אפילו הזזה של בית אחד או פרשנות שגויה של סדר הבתים - מוביל מיידית לייצור Keystream שגוי ולכשל טוטאלי של ההצפנה.

נקודה זו משמעותית במיוחד גם מבחינה קריפטוגרפית: תוקף המנסה לבצע אנליזה על אלגוריתם מוכלל, או על מימוש חומרתי פגום, עשוי לזהות סדירויות או דפוסים שנוצרים עקב טעינת נתונים ב-Big-Endian במקום Little-Endian, או עקב ערבוב בין השניים. כשל כזה עלול להציג בפני התוקף מבנה מטריצה לא תקין או התנהגות לא מתואמת של פונקציית ה-rounds, ובכך לפגוע בתכונות הביזור (diffusion) והבלבול (confusion) שהאלגוריתם תלוי בהן. בפועל, מימוש שגוי של Endianness עשוי להחליש משמעותית את רמת האבטחה של Salsa20 עד כדי הפיכתו לבלתי תקין לחלוטין.

ה.2. תפקידן של המילים הקבועות

תפקידן של המילים הקבועות (constants) – המופיעות במשפט "expend 32-byte k" או "expend 16-byte k" – הוא קריטי להצבת מבנה התחלתי יציב, אחיד ובטוח לכל בלוק. מילים אלה נטענות למטריצת 4×4 בתחילת האלגוריתם ומהוות "עוגנים" קבועים שמייצבים את מבנה הסבבים ומונעים מצב שבו כל הביטוי של האלגוריתם תלוי רק ב-Key וב-Nonce. המילים הקבועות תוכננו כך שייצרו התפשטות מהירה של שינויי ביטים בתוך המטריצה (במיוחד בסבבים הראשונים), ולכן כל פירוש שגוי של סדר הבתים שלהן יגרום להפרת הסימטריה ולפיזור שגוי של ביטים - דבר שמתורגם ל-keystream שגוי או חלש יותר.

בניתוח הכולל של הפרויקט נמצא כי המימוש החומרתי המלא, כולל טעינת המילים הקבועות בפורמט Little-Endian, העניק התאמה מלאה לפלטים הרשמים ולתוצאות הסימולציה. בדיקות זמני FPGA, בדיקות UART ווקטורי הבדיקה החיצוניים הוכיחו שאכיפה עקבית של Little-Endian בכל שלבי ההצפנה - החל מה-Key ועד ל-Nonce וה-Constants - היא גורם מכריע בהבטחת דיוק ההצפנה ורמת החוסן של האלגוריתם.

1. מסקנות וסיכום

במהלך הפרויקט הושגה מטרת העל - מימוש מלא, תקין ויעיל של אלגוריתם ההצפנה Salsa20 ברמת חומרה על גבי רכיבי FPGA שונים. תהליך העבודה, החל מפירוק האלגוריתם לרכיביו הבסיסיים ועד לשילובם בארכיטקטורת RTL מלאה, הוכיח את היתכנות המימוש ואת הדיוק הקריפטוגרפי שלו. הצלבת התוצאות בין סימולציות תוכנה, סימולציות RTL ובדיקות חומרה בזמן אמת הראתה התאמה מלאה לפלטי האלגוריתם המקוריים, גם במקרים רגישים במיוחד כגון טעינת מילים קבועות, ניהול Nonce ויישום עקבי של Little-Endian בכל שלבי ההצפנה. בכך הוכח כי המימוש החומרתי עומד בדרישות התקן הקריפטוגרפי של Salsa20 ומספק Keystream תקין ויציב.

עם זאת, במהלך העבודה עלו מספר נקודות לשיפור ולפיתוח עתידי.

1. ניכר כי נושא ה-Endianness הוא מקור משמעותי לשגיאות פוטנציאליות, ולכן מומלץ לשלב שכבת בדיקות נוספת שתבצע אימות כפול על סדר הבתים.
2. שנית, ניתן לשפר את ביצועי הארכיטקטורה באמצעות pipelining ו-Internal parallelism.
 - באמצעות pipelining

חילוק חישוב האלגוריתם הפנימי לשלבים סדרתיים. במקום להמתין שכל הבלוק יעבור את כל החישוב לפני שמתחילים בבלוק הבא, מתחילים בבלוק הבא כבר בשלב הראשון בעוד שהבלוק הראשון בעיבוד בשלב השני וכו'.

מזכיר פס ייצור במפעל – כל עובד עושה חלק קטן והבלוקים זורמים אחד אחרי השני. היתרון הוא הגדלת מספר הבלוקים שמוצפנים/מפוענחים בשנייה.
 - באמצעות Internal parallelism

ביצוע מספר חישובים במקביל בתוך אותו בלוק או בין בלוקים, במקום לבצע בצורה סדרתית - צורך יותר Area כמו שראינו בפרק ד.1.ב.

היתרון – הקטנת זמן עיבוד לבלוק בודד, כלומר האלגוריתם יסיים את החישוב מהר יותר.
3. בנוסף, המערך הקיים מהווה בסיס מוצק להרחבות עתידיות כגון מימוש גרסאות Salsa20 מקוצרות (20/12/8), מעבר למשפחת ChaCha או שילוב יחידת אימות (MAC) ליצירת מנגנון הצפנה ואימות מלא.

לסיכום, הפרויקט הדגים תהליך מלא - החל מהבנה תאורטית ועד ליישום חומרתי מבצעי - ויצר תשתית אמינה להמשך פיתוח, לייעול וליישומים מעשיים. תרומתו ניכרת הן בפן ההנדסי והן בפן הקריפטוגרפי, והוא מהווה בסיס מוצק להמשך מחקר ומימוש של פרוטוקולי הצפנה מודרניים במערכות משובצות ועתירות ביצועים.

- [1] "Symmetric Encryption vs Asymmetric Encryption: How it Works and Why it's Used," Device Authority. Accessed: Nov. 22, 2025. [Online]. Available: <https://deviceauthority.com/symmetric-encryption-vs-asymmetric-encryption/>
- [2] "What is Stream Cipher and Block Cipher?" Encryption Consulting. Accessed: Nov. 22, 2025. [Online]. Available: <https://www.encryptionconsulting.com/education-center/stream-cipher-and-block-cipher/>
- [3] Wikipedia contributors, "Salsa20," Wikipedia, The Free Encyclopedia. [Online]. Available: <https://en.wikipedia.org/w/index.php?title=Salsa20&oldid=1320012870>
- [4] "Basys 3™ FPGA Board Reference Manual," Digilent.com. Accessed: Nov. 22, 2025. [Online]. Available: https://digilent.com/reference/_media/basys3:%20basys3_rm.pdf
- [5] Intel.com. Accessed: Nov. 22, 2025. [Online]. Available: https://ftp.intel.com/Public/Pub/fpgaup/pub/Intel_Material/Boards/DE10-Lite/DE10_Lite_User_Manual.pdf
- [6] Rohde, Schwarz GmbH, and K. G. Co, "Understanding UART," Rohde-schwarz.com. Accessed: Nov. 22, 2025. [Online]. Available: https://www.rohde-schwarz.com/cz/products/test-and-measurement/essentials-test-equipment/digital-oscilloscopes/understanding-uart_254524.html
- [7] Wikipedia contributors, "Endianness," Wikipedia, The Free Encyclopedia. [Online]. Available: <https://en.wikipedia.org/w/index.php?title=Endianness&oldid=1319620362>
- [8] "Combinational and sequential circuits," GeeksforGeeks. Accessed: Nov. 22, 2025. [Online]. Available: <https://www.geeksforgeeks.org/digital-logic/combinational-and-sequential-circuits/>
- [9] Wikipedia contributors, "Sequential logic," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Sequential_logic&oldid=1280162547

נספחים

קישורים רלוונטיים

- קישור ל- GitHub של הפרויקט :
https://github.com/nisan202/Salsa20_Final_Project/blob/main/README.md
תיקיית "Final Scripts" מכילה את כלל הסקריפטים. עבור כל מימוש, קבצי הפלט שנוצרו באמצעות הסקריפטים ותהליך המימוש נשמרים בתיקייה הפנימית הייעודית של אותו מימוש.
- קישור לסרטון וידאו של מימוש ואינטגרציה מלאה של המערכת על Basis3 :
<https://drive.google.com/file/d/1u9qBgY37mgmlgSJcKttMFOYEu06ENfQ1/view?usp=sharing>
- קישור למצגת הגנת הפרויקט :
https://docs.google.com/presentation/d/1x62xi0_NxIZPHq0ZY0QwzXbiGc-LyVQe/edit?usp=drive_link&ouid=116095597222339870435&rtpof=true&sd=true

נספח א' – פלט פונקציות salsa20 ו-text2bytes

Test Number 0:

Key Vectors:

Key_Hex_List = ['...', '...'.....]

Key_Hex_Bas =

Key_Bin = 0b.....

Nonce Vectors:

Nonce_Hex_List = ['...', '...']

Nonce_Hex_Bas =

Nonce_Bin = 0b.....

Counter Vectors:

Counter_Hex_List = ['...', '...']

Counter_Hex_Bas =

Counter_Bin = 0b.....

Constant Vectors:

Constants_Hex_List = ['0x6b206574' '0x79622d32'
'0x3320646e' '0x61707865']

Constants_Hex_Bas =
6b20657479622d323320646e61707865

Constants_Bin = 0b.....

Initial Matrixs:

Initial_Matrix_Hex =

['61707865' '....' '....' '....']

['....' '3320646e' '....' '....']

['....' '....' '79622d32' '....']

['....' '....' '....' '6b206574']

Initial_Hex_Bas =

Initial_Bas_Bin = 0b.....

Hashing Matrixs:


```

Hashing_Matrix_Hex =
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
Hashing_Hex_Bas = .....
Hashing_Bas_Bin = 0b.....

```

Plain Text Matrixs:

```

Plain_Matrix_Hex =
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
Plain_Hex_Bas = .....
Plain_Text_Bas_Bin = 0b.....

```

Cipher Text Matrixs:

```

Cipher_Matrix_Hex =
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
Cipher_Hex_Bas = .....
Cipher_Text_Bas_Bin = 0b.....

```

Decryption Text Matrixs:

```

Decryption_Matrix_Hex =
    ['....' '....' '....' '....']
    ['....' '....' '....' '....']
    ['....' '....' '....' '.....']
    ['....' '....' '....' '....']
Decryption_Hex_Bas = .....

```

```
Decryption_Text_Bas_Bin = 0b.....
```

```
vectors ready: Encryption and decryption succeeded:
```

נספח ב' - פלט פונקציית perl

```

{
  "Test_ID": "0",
  "Description": "Test Number 0",
  "VECTORS": {
    "PLAIN": [
      "43686170",
      "74657220",
      .....
      "66206e75",
      "6d626572"
    ],
    "KEY": [
      "9eb8ded2",
      "46f68b70",
      .....
      "1ded9c73",
      "c17bbc92"
    ],
    "NONCE": [
      "4e33b0d9",
      "d8011eb6"
    ],
    "COUNTER": [
      "00000000",
      "00000000"
    ],
    "HASH": [
      "6758bdcc",
      "77ad8a66",
      .....

```

```

.....
    "4e39ade3",
    "f3b85673"
],
    "CIPHER": [
        "2430dcbc",
        "03c8f846",
.....
.....
        "2819c396",
        "9eda3301"
    ],
    "DECRYPTION": {
        "normal": [
            "43686170",
            "74657220",
.....
.....
            "66206e75",
            "6d626572"
        ],
        "reversed": [
            "6d626572",
            "66206e75",
.....
.....
            "74657220",
            "43686170"
        ]
    }
},
"REGISTERS": {

```

```

    "PLAIN_TEXT": {
        "encrypt_mode": [
            {
                "address": "0x02",
                "value": "0x6D626572",
                "swap_bytes": "0x7265626D"
            },
            {
                "address": "0x03",
                "value": "0x66206E75",
                "swap_bytes": "0x756E2066"
            },
            {.....}
        ],
        {.....}
    },
    {
        "address": "0x10",
        "value": "0x74657220",
        "swap_bytes": "0x20726574"
    },
    {
        "address": "0x11",
        "value": "0x43686170",
        "swap_bytes": "0x70616843"
    }
],
    "decrypt_mode": [
        {
            "address": "0x02",
            "value": "0x9EDA3301",
            "swap_bytes": "0x0133DA9E"
        }
    ]
}

```

```

    },
    {
        "address": "0x03",
        "value": "0x2819C396",
        "swap_bytes": "0x96C31928"
    },
    {.....
},
{.....
},
{
    "address": "0x10",
    "value": "0x03C8F846",
    "swap_bytes": "0x46F8C803"
},
{
    "address": "0x11",
    "value": "0x2430DCBC",
    "swap_bytes": "0xBCDC3024"
}
]
},
"KEY": [
    {
        "address": "0x12",
        "value": "0xC17BBC92",
        "swap_bytes": "0x92BC7BC1"
    },
    {
        "address": "0x13",
        "value": "0x1DED9C73",
        "swap_bytes": "0x739CED1D"
    }
]

```

```

        },
        {.....
    },
    {.....
        },
        {
            "address": "0x18",
            "value": "0x46F68B70",
            "swap_bytes": "0x708BF646"
        },
        {
            "address": "0x19",
            "value": "0x9EB8DED2",
            "swap_bytes": "0xD2DEB89E"
        }
    ],
    "NONCE": [
        {
            "address": "0x1A",
            "value": "0xD8011EB6",
            "swap_bytes": "0xB61E01D8"
        },
        {
            "address": "0x1B",
            "value": "0x4E33B0D9",
            "swap_bytes": "0xD9B0334E"
        }
    ],
    "COUNTER": [
        {
            "address": "0x1C",
            "value": "0x00000000",

```

```

        "swap_bytes": "0x00000000"
    },
    {
        "address": "0x1D",
        "value": "0x00000000",
        "swap_bytes": "0x00000000"
    }
],
"HASHING": [
    {
        "address": "0x1E",
        "value": "0xF3B85673",
        "swap_bytes": "0x7356B8F3"
    },
    {
        "address": "0x1F",
        "value": "0x4E39ADE3",
        "swap_bytes": "0xE3AD394E"
    },
    {.....
},
{.....
},
    {
        "address": "0x2C",
        "value": "0x77AD8A66",
        "swap_bytes": "0x668AAD77"
    },
    {
        "address": "0x2D",
        "value": "0x6758BDCC",
        "swap_bytes": "0xCCBD5867"
    }
]

```



```

    }
  ],
  "CIPHER": [
    {
      "address": "0x2E",
      "value": "0x9EDA3301",
      "swap_bytes": "0x0133DA9E"
    },
    {
      "address": "0x2F",
      "value": "0x2819C396",
      "swap_bytes": "0x96C31928"
    },
    {.....}
  ],
  {.....}
},
{
  "address": "0x3C",
  "value": "0x03C8F846",
  "swap_bytes": "0x46F8C803"
},
{
  "address": "0x3D",
  "value": "0x2430DCBC",
  "swap_bytes": "0xBCDC3024"
}
]
}
},

```

נספח ג' - XDC Basys3

```
#####

# Clocks #

#####

# The clock set on the board - 100MHz

create_clock -period 10.000 -name salsa_top_100mhz_clk -
waveform {0.000 5.000} [get_ports salsa_top_100mhz_clk]

# The clock after PLL - 80MHz

create_generated_clock -name salsa_top_80mhz_pll_clk -source
[get_ports salsa_top_100mhz_clk] -divide_by 5 -multiply_by 4
[get_pins salsa_top_xilinx_ip_pll/clk_out1]

#####

# Input Delay #

#####

set_input_delay -clock salsa_top_80mhz_pll_clk -add_delay
0.000 [get_ports {salsa_top_rst_n rs232_rx_top}]

#####

# Output Delay #

#####

set_output_delay -clock salsa_top_80mhz_pll_clk -add_delay
0.000 [get_ports {rs_232_tx_top hash_done_sticky_top
xor_valid_sticky_top salsa_top_pll_locked}]

set_output_delay -clock salsa_top_80mhz_pll_clk -add_delay
0.000 [get_ports {anode_set_top* dp_set_top seven_seg_top*}]

#####

# Pin Locations and Voltages #

#####

# clk

set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33}
[get_ports salsa_top_100mhz_clk]
```

```

# rst_n active low rst - switch --> SW15 (R2)
set_property -dict {PACKAGE_PIN R2 IOSTANDARD LVCMOS33}
[get_ports salsa_top_rst_n]

# rx
set_property -dict {PACKAGE_PIN B18 IOSTANDARD LVCMOS33}
[get_ports rs232_rx_top]

# tx
set_property -dict {PACKAGE_PIN A18 IOSTANDARD LVCMOS33}
[get_ports rs_232_tx_top]

# sticky signals - LED (LD0 & LD1)
set_property -dict {PACKAGE_PIN U16 IOSTANDARD LVCMOS33}
[get_ports hash_done_sticky_top]
set_property -dict {PACKAGE_PIN E19 IOSTANDARD LVCMOS33}
[get_ports xor_valid_sticky_top]

# pll locked signal - LED (LD2)
set_property -dict {PACKAGE_PIN U19 IOSTANDARD LVCMOS33}
[get_ports salsa_top_pll_locked]

##### SSD #####

# Anode
set_property -dict {PACKAGE_PIN U2 IOSTANDARD LVCMOS33}
[get_ports {anode_set_top[0]}]
set_property -dict {PACKAGE_PIN U4 IOSTANDARD LVCMOS33}
[get_ports {anode_set_top[1]}]
set_property -dict {PACKAGE_PIN V2 IOSTANDARD LVCMOS33}
[get_ports {anode_set_top[2]}]
set_property -dict {PACKAGE_PIN W4 IOSTANDARD LVCMOS33}
[get_ports {anode_set_top[3]}]

# DP
set_property -dict {PACKAGE_PIN V7 IOSTANDARD LVCMOS33}
[get_ports dp_set_top]

```

```

# 7 Segment

set_property -dict {PACKAGE_PIN W7 IOSTANDARD LVCMOS33}
[get_ports {seven_seg_top[0]}]

set_property -dict {PACKAGE_PIN W6 IOSTANDARD LVCMOS33}
[get_ports {seven_seg_top[1]}]

set_property -dict {PACKAGE_PIN U8 IOSTANDARD LVCMOS33}
[get_ports {seven_seg_top[2]}]

set_property -dict {PACKAGE_PIN V8 IOSTANDARD LVCMOS33}
[get_ports {seven_seg_top[3]}]

set_property -dict {PACKAGE_PIN U5 IOSTANDARD LVCMOS33}
[get_ports {seven_seg_top[4]}]

set_property -dict {PACKAGE_PIN V5 IOSTANDARD LVCMOS33}
[get_ports {seven_seg_top[5]}]

set_property -dict {PACKAGE_PIN U7 IOSTANDARD LVCMOS33}
[get_ports {seven_seg_top[6]}]


#####

# Configuration Settings #

#####

set_property CFGBVS VCCO [current_design]

set_property CONFIG_VOLTAGE 3.3 [current_design]

```

נספח ד' - SDC DE10-Lite

```
#####

# Clocks #

#####

#The clock set on the board - 50MHz

create_clock -name {salsa_top_50mhz_clk} -period "20.000"
[get_ports salsa_top_50mhz_clk]

#uncertainty

derive_clock_uncertainty

#####

# Input Delay #

#####

set_input_delay -clock {salsa_top_50mhz_clk} -max 0.000
[get_ports {salsa_top_rst_n rs232_rx_top}]

set_input_delay -clock {salsa_top_50mhz_clk} -min 0.000
[get_ports {salsa_top_rst_n rs232_rx_top}]

#####

# Output Delay #

#####

set_output_delay -clock {salsa_top_50mhz_clk} -max 0.000
[get_ports {rs_232_tx_top hash_done_sticky_top
xor_valid_sticky_top}]

set_output_delay -clock {salsa_top_50mhz_clk} -min 0.000
[get_ports {rs_232_tx_top hash_done_sticky_top
xor_valid_sticky_top}]

set_output_delay -clock {salsa_top_50mhz_clk} -max 0.000
[get_ports {dp_set_top seven_seg_top*}]

set_output_delay -clock {salsa_top_50mhz_clk} -min 0.000
[get_ports {dp_set_top seven_seg_top*}]
```

נספח ה' - QSF DE10-Lite

```

# ----- #
#
#Copyright (C) 2024 Intel Corporation. All rights reserved.
#Your use of Intel Corporation's design tools, logic
functions
#and other software and tools, and any partner logic
#functions, and any output files from any of the foregoing
#(including device programming or simulation files), and any
#associated documentation or information are expressly
subject
#to the terms and conditions of the Intel Program License
#Subscription Agreement, the Intel Quartus Prime License
Agreement,
#the Intel FPGA IP License Agreement, or other applicable
license
#agreement, including, without limitation, that your use is
for
#the sole purpose of programming logic devices manufactured
by
#Intel and sold by Intel or its authorized distributors.
Please
#refer to the applicable agreement for further details, at
#https://fpgasoftware.intel.com/eula.
#
# ----- #
# -----
#
#Quartus Prime
#Version 23.1std.1 Build 993 05/14/2024 SC Lite Edition
#Date created = 22:51:07 November 23, 2025
#
# ----- #
# -----
#

```

```

#Notes:
#
(1 #The default values for assignments are stored in the
file:
#         top_salsa_intel_00_assignment_defaults.qdf
#         #If this file doesn't exist, see file:
#         assignment_defaults.qdf
#
(2 #Intel recommends that you do not modify this file. This
#file is updated automatically by the Quartus Prime
software
#and any changes you make may be lost or overwritten.
#
----- #
# -----

set_global_assignment -name FAMILY "MAX 10"
set_global_assignment -name DEVICE 10M50DAF484C7G
set_global_assignment -name TOP_LEVEL_ENTITY top_salsa_intel
set_global_assignment -name ORIGINAL_QUARTUS_VERSION 23.1STD.1
set_global_assignment -name PROJECT_CREATION_TIME_DATE
"22:51:07 NOVEMBER 23, 2025"
set_global_assignment -name LAST_QUARTUS_VERSION "23.1std.1
Lite Edition"
set_global_assignment -name PROJECT_OUTPUT_DIRECTORY
output_files
set_global_assignment -name MIN_CORE_JUNCTION_TEMP 0
set_global_assignment -name MAX_CORE_JUNCTION_TEMP 85
set_global_assignment -name DEVICE_FILTER_PACKAGE FBGA
set_global_assignment -name DEVICE_FILTER_PIN_COUNT 484
set_global_assignment -name DEVICE_FILTER_SPEED_GRADE 7
set_global_assignment -name ERROR_CHECK_FREQUENCY_DIVISOR 256
set_global_assignment -name EDA_SIMULATION_TOOL "Questa Intel
FPGA (Verilog)"

```

```

set_global_assignment -name EDA_TIME_SCALE "1 ps" -section_id
eda_simulation

set_global_assignment -name EDA_OUTPUT_DATA_FORMAT "VERILOG
HDL" -section_id eda_simulation

set_global_assignment -name EDA_GENERATE_FUNCTIONAL_NETLIST
OFF -section_id eda_board_design_timing

set_global_assignment -name EDA_GENERATE_FUNCTIONAL_NETLIST
OFF -section_id eda_board_design_symbol

set_global_assignment -name EDA_GENERATE_FUNCTIONAL_NETLIST
OFF -section_id eda_board_design_signal_integrity

set_global_assignment -name EDA_GENERATE_FUNCTIONAL_NETLIST
OFF -section_id eda_board_design_boundary_scan

set_global_assignment -name PARTITION_NETLIST_TYPE SOURCE -
section_id Top

set_global_assignment -name
PARTITION_FITTER_PRESERVATION_LEVEL PLACEMENT_AND_ROUTING -
section_id Top

set_global_assignment -name PARTITION_COLOR 16764057 -
section_id Top

set_global_assignment -name POWER_PRESET_COOLING_SOLUTION "23
MM HEAT SINK WITH 200 LFPM AIRFLOW"

set_global_assignment -name POWER_BOARD_THERMAL_MODEL "NONE
(CONSERVATIVE)"


#I added that manually
#####

#Pin Locations and Voltages#
#####

#clk -> P11 = max10_clk1_50
set_location_assignment PIN_P11 -to salsa_top_50mhz_clk
set_instance_assignment -name IO_STANDARD "3.3-V LVTTTL" -to
salsa_top_50mhz_clk


#rst_n active low rst - switch --> SW9 (F15)
set_location_assignment PIN_F15 -to salsa_top_rst_n

```



```
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
salsa_top_rst_n
```

```
#rx - Arduino IO0 PIN_AB5 => connect to tx on FTDI.
set_location_assignment PIN_AB5 -to rs232_rx_top
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
rs232_rx_top
```

```
#tx - Arduino IO1 PIN_AB6 => connect to rx on FTDI.
set_location_assignment PIN_AB6 -to rs_232_tx_top
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
rs_232_tx_top
```

```
#sticky signals - LEDs
```

```
#LD0
set_location_assignment PIN_A8 -to hash_done_sticky_top
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
hash_done_sticky_top
```

```
#LD1
set_location_assignment PIN_A9 -to xor_valid_sticky_top
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
xor_valid_sticky_top
```

```
----- #SSD-----
```

```
#DP
set_location_assignment PIN_L19 -to dp_set_top
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
dp_set_top
```

```
7 #Segment - HEX5 - the left one on board
```

```
set_location_assignment PIN_J20 -to seven_seg_top[0]
```

```

set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
seven_seg_top[0]

set_location_assignment PIN_K20 -to seven_seg_top[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
seven_seg_top[1]

set_location_assignment PIN_L18 -to seven_seg_top[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
seven_seg_top[2]

set_location_assignment PIN_N18 -to seven_seg_top[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
seven_seg_top[3]

set_location_assignment PIN_M20 -to seven_seg_top[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
seven_seg_top[4]

set_location_assignment PIN_N19 -to seven_seg_top[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
seven_seg_top[5]

set_location_assignment PIN_N20 -to seven_seg_top[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
seven_seg_top[6]

set_global_assignment -name SDC_FILE "../Code_files/6.
xdc_files/DE10Lite_salsa_top.sdc"

set_global_assignment -name VERILOG_INCLUDE_FILE
../Code_files/3.Defines_file/salsa_defines.vh

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/quarter_round.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/hash_interative_with_d_r_c
.sv

```

```

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/d_r_c_round_interative.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/constant_select.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/top_salsa_intel.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/salsa_xor_regs.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/salsa_uart_tx.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/salsa_rx.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/salsa_registers.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/salsa_hash_interative_wrap
per.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/salsa_fsm.sv

set_global_assignment -name SYSTEMVERILOG_FILE
../Code_files/1.Verilog_Design_Code/hex_to_SSD_Intel.sv

set_instance_assignment -name PARTITION_HIERARCHY
root_partition -to | -section_id Top

```